

Maintaining Leiden Communities in Large Dynamic Graphs (Full Version)

Chunxu Lin
CUHK-Shenzhen
Shenzhen, China
chunxulin1@link.cuhk.edu.cn

Yumao Xie
CUHK-Shenzhen
Shenzhen, China
yumaoxie@link.cuhk.edu.cn

Yixiang Fang
CUHK-Shenzhen
Shenzhen, China
fangyixiang@cuhk.edu.cn

Yongmin Hu
ByteDance Inc.
Hangzhou, China
huyongmin@bytedance.com

Yingqian Hu
ByteDance Inc.
Hangzhou, China
huyingqian@bytedance.com

Cheng Chen
ByteDance Inc.
Singapore
chencheng.sg@bytedance.com

Abstract—As a well-known community detection algorithm, Leiden has been widely used in various scenarios such as large language model (LLM) generation, anomaly detection, and biological analysis. In these scenarios, the graphs are often large and dynamic, where vertices and edges are inserted and deleted frequently, so it is costly to obtain the updated communities by Leiden from scratch when the graph has changed. Recently, one work has attempted to study how to maintain Leiden communities in the dynamic graph, but it lacks a detailed theoretical analysis, and its algorithms are inefficient for large graphs. To address these issues, in this paper, we first theoretically show that the existing algorithms are relatively unbounded via the boundedness analysis (a powerful tool for analyzing incremental algorithms on dynamic graphs), and also analyze the memberships of vertices in communities when the graph changes. Based on theoretical analysis, we develop a novel efficient maintenance algorithm, called *Hierarchical Incremental Tree Leiden* (HIT-Leiden), which effectively reduces the range of affected vertices by maintaining the connected components and hierarchical community structures. Comprehensive experiments on various datasets demonstrate the superior performance of HIT-Leiden. In particular, it achieves speedups of up to five orders of magnitude over existing methods.

Index Terms—Incremental graph algorithms, community detection, Leiden algorithm.

I. INTRODUCTION

As one of the fundamental measures in network science, modularity [1] effectively measures the strength of division of a network into modules (also called communities). Essentially, it captures the difference between the actual number of edges within a community and the expected number of such edges if connections were random. By maximizing the modularity of a graph, it can reveal all the communities in the graph. In Fig. 1(a), for example, by maximizing the modularity of the graph, we can obtain two communities C_1 and C_2 . As shown in the literature [2], [3], graph communities have found a wide range of applications in recommendation systems, social marketing, and biological analysis.

One of the most popular community detection (CD) algorithms that uses modularity maximization is Louvain [4], which partitions a graph into disjoint communities. As shown in Fig. 2(a), Louvain employs an iterative process with each it-

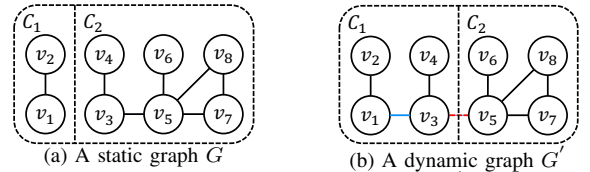


Fig. 1. Illustrating community maintenance, where (v_1, v_3) is a newly inserted edge and (v_3, v_5) is a newly deleted edge.

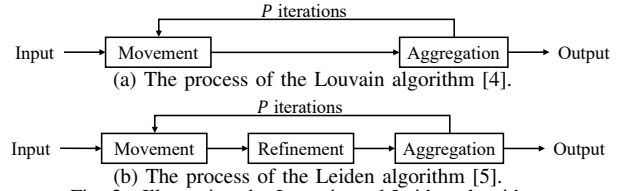


Fig. 2. Illustrating the Louvain and Leiden algorithms.

eration having two phases, called **movement** and **aggregation**, to adjust the community structure and improve modularity. Specifically, in the movement phase, each vertex is relocated to a suitable community to maximize the modularity of the graph. In the aggregation phase, all the vertices belonging to the same community are merged into a supervertex to form a supergraph for the next iteration. Since a supervertex corresponds to a set of vertices, the communities of a graph naturally form a tree-like hierarchical structure. In practice, to balance modularity gains against the running time, users often limit Louvain to P iterations, where P is a pre-defined parameter.

Despite its popularity, Louvain may produce communities that are internally disconnected. This typically occurs during the movement phase, where a vertex that serves as a bridge within a community may be moved to a different community that has stronger connections, thereby breaking the connectivity of the original community. To overcome this issue, Traag et al. [5] proposed the *Leiden* algorithm¹, which introduces an additional phase, called **refinement**, between the movement and aggregation phases, as shown in Fig. 2(b). Specifically, during the refinement phase, vertices explore merging with their neighbors within the same community to form sub-

¹As of June 2026, Leiden has received over 7,000 citations according to Google Scholar.

communities. By adding this additional phase, Leiden produces communities with higher quality than Louvain, since its communities well preserve the connectivity.

As shown in the literature, Leiden has recently received plenty of attention because of its applications in many areas, including large language model (LLM) generation [6], anomaly detection [7], and biological analysis [8]. For example, Microsoft has recently developed Graph-RAG [6], a retrieval-augmented generation (RAG) method that enhances prompts by searching external knowledge to improve the accuracy and trustworthiness of LLM generation, and builds a hierarchical index by using the communities detected by Leiden. As another example, Liu et al. introduced eRiskComm [9], a community-based fraud detection system that assists regulators in identifying high-risk individuals from social networks by using Louvain to partition communities, and Leiden can be naturally applied in this context.

In the aforementioned application scenarios, the graphs often evolve frequently over time, with many insertions and deletions of vertices and edges. We highlight two representative such tasks in ByteDance. (1) *Anomaly detection*: This task’s pipeline operates on the graph with 10^8 vertices and 2×10^9 edges, where recomputing Leiden communities from scratch introduces a delay of several hours. (2) *Social recommendation*: This task works on a graph with 10^8 vertices and 4×10^9 edges. However, the online monitoring and operational workflows require incremental maintenance for batches of roughly 10^5 edge updates within a few minutes. Full recomputation of Leiden communities incurs hour-level delays, while product requirements demand minute-level updates for update batches of this scale. Hence, it is strongly desirable to develop efficient algorithms for maintaining the up-to-date Leiden communities in large dynamic graphs.

Prior works. To maintain Louvain communities in dynamic graphs, several algorithms have been developed, such as DF-Louvain [10], Delta-Screening [11], DynaMo [12], and Batch [13]. However, little attention has been paid to maintaining Leiden communities. To the best of our knowledge, [14] is the only work that achieves this. It first uses some optimizations for the first iteration of DF-Leiden, and then invokes the original Leiden algorithm for the remaining iterations, as depicted in Fig. 3(a). Following the optimized movement phase (opt-movement), the refinement phase in DF-Leiden separates communities affected by edge or vertex changes into multiple sub-communities, while leaving unchanged communities as single sub-communities. The aggregation phase remains identical to that of the Leiden algorithm. After constructing the aggregated graph, the standard Leiden algorithm is applied to complete the remaining CD process. The authors have also developed two variants of DF-Leiden, called ND-Leiden and DS-Leiden, by using different optimizations for the movement phase of the first iteration. Nevertheless, there is a lack of detailed theoretical analysis for these algorithms, and they are inefficient for large graphs with few changes.

Our contributions. To address the above limitations, we first theoretically analyze the time cost of existing algorithms for maintaining Leiden communities and theoretically show

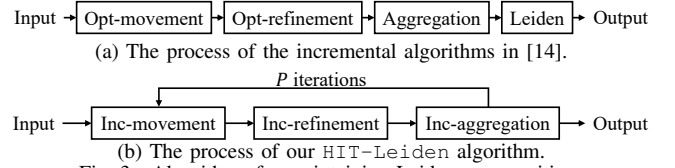


Fig. 3. Algorithms for maintaining Leiden communities.

that they are relatively unbounded via the boundedness analysis, which is a powerful tool for analyzing the time complexity of incremental algorithms on dynamic graphs. We further analyze the membership of vertices in communities and sub-communities when the graph edges change, and observe that the procedure for maintaining these memberships generalizes naturally to all the supergraphs generated by Leiden. The above analysis not only lays a solid foundation for us to comprehend existing algorithms but also offers us opportunities to improve upon them.

Based on the above analyses, we develop a novel efficient maintenance algorithm, called Hierarchical Incremental Tree Leiden (HIT-Leiden), which effectively reduces the range of affected vertices by maintaining the connected components and hierarchical community structures. As depicted in Fig. 3(b), HIT-Leiden is an iterative algorithm with each iteration having three key phases, namely incremental movement, incremental refinement, and incremental aggregation, abbreviated as inc-movement, inc-refinement, and inc-aggregation, respectively. More specifically, inc-movement extends the movement phase from [14] by incorporating hierarchical community structures [5]. Unlike prior approaches, it operates on a supergraph where each supervertex represents a sub-community, focusing on hierarchical dependencies between communities and their nested substructures. Inspired by the key technique of maintaining the connected components in dynamic graphs [15], inc-refinement maintains sub-communities by using tree-based structures to efficiently track changes in sub-communities. Inc-aggregation updates the supergraph by computing structural changes based on the outputs of the previous two phases.

We have evaluated HIT-Leiden on several large-scale real-world dynamic graph datasets. The experimental results show that our algorithm achieves comparable community quality with the state-of-the-art algorithms for maintaining Leiden communities, while achieving speedups of up to five orders of magnitude over DF-Leiden. In addition, we have deployed our algorithm in real-world applications at ByteDance.

Outline. We first review related work in Section II. We then formally introduce some preliminaries, including the Leiden algorithm and problem definition in Section III, provide some theoretical analysis in Section IV, and present our proposed HIT-Leiden algorithm in Section V. Finally, we present the experimental results in Section VI and conclude in Section VII.

II. RELATED WORK

In this section, we first review the existing works of CD for both static and dynamic graphs. We simply classify these works as modularity and other metrics-based CD methods.

• **Modularity-based CD.** Modularity-based CD methods aim to partition a graph such that communities exhibit high internal connectivity relative to a null model. Among these methods, Louvain [4] is the most popular one due to its high efficiency and scalability, as shown in some comparative analyses [16], [17]. Leiden [5] improves upon Louvain by resolving the problem of disconnected communities, yielding higher-quality results with comparable runtime. Other methods employ different modularity heuristics [18], incorporate simulated annealing [19], spectral techniques [20], or evolutionary strategies [21]. Recent neural approaches have integrated modularity objectives into deep learning models [22]–[24], enhancing representation learning for CD.

Besides, some recent works have studied how to incrementally maintain modularity-based communities when the graph is changed. Aynaud et al. [25] proposed one of the earliest approaches by reusing previous community assignments to warm-start the Louvain algorithm. Subsequent works extended this idea to both Louvain [10]–[12] and Leiden [14], incorporating mechanisms such as edge-based impact screening or localized modularity updates. Nevertheless, the existing algorithms for maintaining Leiden communities lack in-depth theoretical analysis, and their practical efficiency is poor. Other methods based on modularity, including extensions to spectral clustering [26], multi-step CD [27], and label propagation-based methods [28] have been studied on dynamic graphs.

• **Other metrics-based CD.** Beyond modularity, various CD methods have been developed for different optimization purposes, such as similarity, statistical inference, spectral clustering, and neural networks. The similarity-based methods like SCAN [29]–[31] identify dense regions from the graph via structural similarity. Statistical inference approaches, including stochastic block models [32]–[34], infer communities by fitting generative probabilistic models to observed networks. Spectral clustering methods [35], [36] exploit the eigenstructure of graph Laplacians to group nodes with similar structural roles. Deep learning-based methods for CD have recently gained traction. Graph convolutional networks [37]–[39], and graph attention networks [40]–[42] have demonstrated strong performance in learning expressive node embeddings for CD tasks. For more details, please refer to recent survey papers of CD [2], [3].

Besides, many of the above methods have also been extended for dynamic graphs. Ruan et al. [43] and Zhang et al. [44] have studied structural graph clustering on dynamic graphs, which is based on structural similarity. Temporal spectral methods [45] and dynamic stochastic block models [46] enable statistical modeling of evolving community structures over time. Recent deep learning approaches also support dynamic CD through mechanisms such as dynamic embeddings [47] and contrastive learning [48]. These models capture temporal dependencies and structural evolution.

III. PRELIMINARIES

In this section, we first briefly recap the original Leiden procedure, then briefly introduce the maintenance of Leiden communities problem, and finally introduce the key notations.

Algorithm 1: Leiden algorithm [49], [50]

Input: $G, f(\cdot), P, \gamma$

Output: Updated $f(\cdot)$

```

1  $G^1 \leftarrow G, f^1(\cdot) \leftarrow f(\cdot);$ 
2 for  $p = 1$  to  $P$  do
3    $f^p(\cdot) \leftarrow \text{Move}(G^p, f^p(\cdot), \gamma);$ 
4    $s^p(\cdot) \leftarrow \text{Refine}(G^p, f^p(\cdot), \gamma);$ 
5   if  $p < P$  then
6      $G^{p+1}, f^{p+1}(\cdot) \leftarrow \text{Aggregate}(G^p, f^p(\cdot), s^p(\cdot));$ 
7 Update  $f(\cdot)$  using  $s^1(\cdot), \dots, s^P(\cdot);$ 
8 return  $f(\cdot);$ 
```

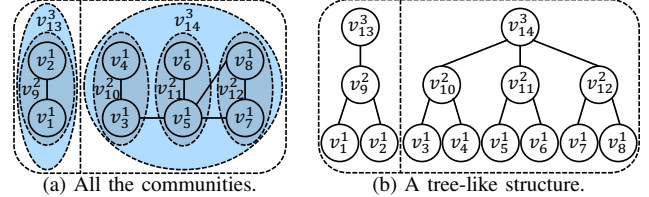


Fig. 4. The process of Leiden for the graph G in Fig. 1(a).

A. Leiden algorithm

Originally, the Leiden algorithm was designed for static graphs. It mainly consists of three phases: *movement*, *refinement*, and *aggregation*, which are executed iteratively. Algorithm 1 [49], [50] presents the Leiden algorithm following the process in Fig. 2(b). Starting from the original graph $G^1 = G$ and an initial community mapping $f(\cdot)$, Leiden iterates for P rounds (lines 1-2) where P is a pre-defined parameter that users often set in practice to balance modularity gains against runtime. γ is a hyperparameter to control the scale of communities [51]. In each round p , the *movement* phase relocates vertices (or supervertices) to improve modularity, producing an updated mapping $f^p(\cdot)$ from the vertices (or supervertices) in this iteration to communities (line 3). The *refinement* phase splits each community into connected sub-communities, producing a sub-community mapping $s^p(\cdot)$ from the supervertices in this iteration to the supervertices in the next iteration (line 4). The *aggregation* phase contracts each sub-community into a supervertex to form the next-level supergraph G^{p+1} (lines 5-6). After P rounds, the final partition $f(\cdot)$ of G is obtained by composing the mappings across levels.

The overall time complexity of Algorithm 1 is $O(P \cdot (|V| + |E|))$, since each iteration costs $O(|V| + |E|)$ time [50], where V and E are the vertex set and edge set of G , respectively.

Example 1. Fig. 4(a) depicts the process of Leiden with $P=3$ for the graph in Fig. 1(a). Let v_i^p denote the supervertex in the p -th iteration of Leiden. It generates three levels of supergraphs: G^1 , G^2 , and G^3 , with $G^1 = G$. The vertices of these supergraphs form a tree-like structure, as shown in Fig. 4(b).

Take the first iteration as an example depicted in Fig. 5. In the movement phase, it generates three communities $C_1 = \{v_1^1, v_2^1\}$, $C_2 = \{v_5^1, v_6^1, v_7^1, v_8^1\}$ and $C_3 = \{v_3^1, v_4^1\}$. In the refinement phase, C_2 is split into two sub-communities

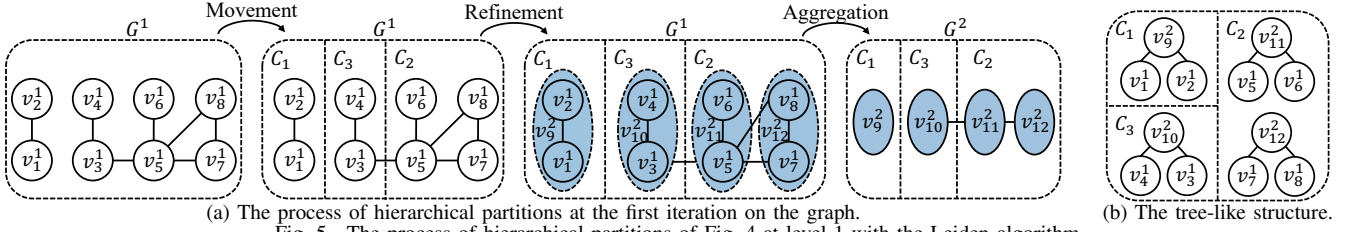


Fig. 5. The process of hierarchical partitions of Fig. 4 at level-1 with the Leiden algorithm.

$v_{11}^2 = \{v_5^1, v_6^1\}$ and $v_{12}^2 = \{v_7^1, v_8^1\}$, and C_1 and C_3 are unchanged. In the aggregation phase, all vertices are aggregated into supervertices based on their sub-community memberships, resulting in G^2 .

B. Maintaining Leiden communities in dynamic graphs

For simplicity, we call the communities generated by the Leiden algorithm *Leiden communities*. The goal of maintaining Leiden communities is to refresh the communities after each graph update. We now formally introduce the problem studied in this paper.

Problem 1 (Maintenance of Leiden communities [14]). *Given a graph G with its Leiden communities $\mathbb{C}$, and some edge updates ΔG , return the updated Leiden communities after applying ΔG to G .*

As shown in Algorithm 1, the Leiden algorithm has three phases. As a result, to maintain Leiden communities when the graph is updated, we can follow the same three-phase procedure, but only make updates in a small affected subgraph: (1) *Movement phase*: we update the community assignments for affected vertices to improve modularity; (2) *Refinement phase*: we restore the connectivity guarantees by splitting and merging affected parts inside each community; (3) *Aggregation phase*: we update the supergraph and propagate structural changes to higher levels when needed.

However, this task is challenging for two reasons. First, the refinement phase enforces intra-community connectivity, which requires efficiently tracking connectivity changes under edge insertions or deletions. Second, because the refinement and aggregation phases are tightly coupled, even a small local update may cascade across multiple hierarchical levels, leading to unstable and potentially large maintenance costs.

C. Notation definition

We consider an **undirected and weighted graph** $G = (V, E)$, where V and E are the sets of vertices and edges, respectively. Each vertex v 's neighbor set is denoted by $N(v)$. Each edge (v_i, v_j) is associated with a positive weight $w(v_i, v_j) > 0$. The degree of v_i is given by $d(v_i) = \sum_{v_j \in N(v_i)} w(v_i, v_j)$. Denote by m the total weight of all edges in G , i.e., $m = \sum_{(v_i, v_j) \in E} w(v_i, v_j)$. For convenience, Table I provides a quick reference for frequently used symbols.

Given a graph $G = (V, E)$, the CD process aims to partition all the vertices of V into some disjoint sets $\mathbb{C}$, each of which is called a community, corresponding to a set of vertices that are densely connected. This process can be modeled as a mapping function $f(\cdot) : V \rightarrow \mathbb{C}$, such that each v belongs to a community $f(v)$ of the partition $\mathbb{C}$. For each vertex v ,

TABLE I
FREQUENTLY USED NOTATIONS AND THEIR MEANINGS.

Notation	Meaning
$G = (V, E)$	A graph with vertex set V and edge set E
$N(v), N_2(v)$	The vertex v 's 1- and 2-hop neighbor sets, resp.
$w(v_i, v_j)$	The weight of edge between v_i and v_j
$d(v)$	The weighted degree of vertex v
m	The total weight of all edges in G
$\mathbb{C}$	A set of communities forming a partition of G
Q	The modularity of the graph G with partition $\mathbb{C}$
$G^p = (V^p, E^p)$	The supergraph in the p -th iteration of Leiden
$\Delta Q(v \rightarrow C', \gamma)$	Modularity gain by moving v from C to C' with γ
$f(\cdot) : V \rightarrow \mathbb{C}$	A mapping from vertices to communities
$f^p(\cdot) : V^p \rightarrow \mathbb{C}$	A mapping from supervertices to communities
$s^p(\cdot)$	A mapping from supervertices in p -th level to supervertices in $(p+1)$ -th level (sub-communities)
V^p, E^p	A set of supervertices and superedges in p -th level
ΔG	The set of changed edges in the dynamic graph

the total weight between v and a community C is denoted by $w(v, C) = \sum_{v' \in N(v) \cap C} w(v, v')$.

As a well-known CD metric, the modularity measures the difference between the actual number of edges in a community and the expected number of such edges.

Definition 1 (Modularity [4]). *Given a graph $G = (V, E)$ and a community partition $\mathbb{C}$ over V , the modularity $Q(G, \mathbb{C}, \gamma)$ of the graph G with the partition $\mathbb{C}$ is defined as:*

$$Q(G, \mathbb{C}, \gamma) = \sum_{C \in \mathbb{C}} \left(\frac{1}{2m} \sum_{v \in C} w(v, C) - \gamma \left(\frac{d(C)}{2m} \right)^2 \right), \quad (1)$$

where $d(C)$ is the total degree of all vertices in a community C , and $\gamma > 0$ is a hyperparameter.

Note that the parameter $\gamma > 0$ controls the granularity of the detected communities [51]. A higher γ favors smaller, finer-grained communities. In practice, γ is often set to 0.5, 1, 4, or 32, as shown in [52]. Besides, to guide community updates, the concept of modularity gain is often used to capture the changed modularity when a vertex is moved from one community to another.

Definition 2 (Modularity gain [4]). *Given a graph G , a partition $\mathbb{C}$, and a vertex v that belongs to a community C , the modularity gain of moving v from C to another community C' is defined as:*

$$\Delta Q(v \rightarrow C', \gamma) = \frac{w(v, C') - w(v, C)}{m} + \frac{\gamma \cdot d(v) \cdot (d(C) - d(v) - d(C'))}{2m^2}. \quad (2)$$

In this paper, we focus on the dynamic graph with insertions and deletions of both vertices and edges. Since a vertex insertion (resp. deletion) can be modeled as a sequence of edge

insertions (resp. deletions), we simply focus on edge changes. Given a set of edge changes ΔG to a graph $G = (V, E)$, we obtain an updated graph $G' = (V', E')$. Since there are two types of edge updates, we let $\Delta G = \Delta G_+ \cup \Delta G_-$, where $\Delta G_+ = E' \setminus E$ and $\Delta G_- = E \setminus E'$ denote the sets of inserted and deleted edges, respectively. We denote updated edges $(v_i, v_j, \alpha) \in \Delta G_+$ and $(v_i, v_j, -\alpha) \in \Delta G_-$, where α is positive, i.e., $\alpha > 0$. We use $G \oplus \Delta G$ to denote applying ΔG to G , yielding an updated graph G' .

IV. THEORETICAL ANALYSIS OF LEIDEN

In this section, we first analyze the boundedness of existing algorithms, then study how vertex behavior impacts community structure under graph updates, and extend our analysis to supergraphs. For lack of space, all the proofs of lemmas are shown in the appendix of the full version [53] of this paper.

A. Boundedness analysis

We first introduce some concepts related to boundedness.

- **Notation.** Let Θ denote the CD query applied to a graph G , where $\Theta(G) = \mathbb{C}$ is the set of detected communities. The new graph is $G \oplus \Delta G$, and the updated community is $\Theta(G \oplus \Delta G)$. We denote the output difference as $\Delta \mathbb{C}$, where $\Theta(G \oplus \Delta G) = \Theta(G) \oplus \Delta \mathbb{C}$.

- **Concepts of boundedness.** The notion of boundedness [54] evaluates the effectiveness of an incremental algorithm using the metric CHANGED, defined as $\text{CHANGED} = \Delta G + \Delta \mathbb{C}$, which leads to $|\text{CHANGED}| = |\Delta G| + |\Delta \mathbb{C}|$.

Definition 3 (Boundedness [54], [55]). *An incremental algorithm is bounded if its computational cost can be expressed as a polynomial function of $|\text{CHANGED}|$ and $|\Theta|$. Otherwise, it is unbounded.*

- **Concepts of relative boundedness.** In real-world dynamic graphs, $|\text{CHANGED}|$ is often small, yet some unbounded algorithms can be solved in polynomial time using measures comparable to $|\text{CHANGED}|$, making these algorithms feasible. To assess these incremental algorithms effectively, Fan et al. [55] introduced the concept of relative boundedness, which leverages a more refined cost model called the affected region. Let AFF denote the affected part, the region of the graph actually processed by the incremental algorithm.

Definition 4 (AFF [55]). *Given a graph G , a query Θ , and the input update ΔG to G , AFF signifies the cost difference of the static algorithm between computing $\Theta(G)$ and $\Theta(G \oplus \Delta G)$.*

Unlike CHANGED, AFF captures the concrete portion of the graph touched by an incremental algorithm, providing a tighter bound on its computational cost. This leads to the following definition.

Definition 5 (Relative boundedness [55]). *An incremental graph algorithm is relatively bounded to the static algorithm if its cost is polynomial in $|\Theta|$ and $|\text{AFF}|$.*

We now analyze the boundedness of existing incremental Leiden algorithms.

TABLE II
INCREMENTAL LEIDEN ALGORITHMS

Method	Time complexity	Relative boundedness
ND-Leiden [14]	$O\left(\sum_{p=1}^P (V^p + E^p)\right)$	$\times$
DS-Leiden [14]	$O\left(\sum_{p=1}^P (V^p + E^p)\right)$	$\times$
DF-Leiden [14]	$O\left(\sum_{p=1}^P (V^p + E^p)\right)$	$\times$
HIT-Leiden	$O(N_2(\text{CHANGED}) + N_2(\text{AFF}))$	$\checkmark$

Theorem 1. *When processing an edge deletion or insertion, the incremental Leiden algorithms proposed in [14] all cost $O\left(\sum_{p=1}^P (|V^p| + |E^p|)\right)$.*

By Theorem 1, the existing algorithms for maintaining Leiden communities are both unbounded and relatively unbounded, as shown in Table II. They are very costly for large graphs, even for small updates. Next, we review the properties of Leiden and then identify AFF of Leiden.

B. Vertex optimality and subpartition γ -density

As shown in the literature [4], [5], if $s^P(\cdot) = f^P(\cdot)$ after P iterations, Leiden is guaranteed to satisfy the following two properties:

- **Vertex optimality:** All the vertices are vertex optimal.
- **Subpartition γ -density:** All the communities are subpartition γ -dense.

To design an efficient and effective maintenance algorithm for Leiden communities, we analyze the behaviors of vertices and communities when the graph changes as follows.

- **Analysis of vertex optimality.** We begin with a key concept.

Definition 6 (Vertex optimality [4]). *A community $C \in \mathbb{C}$ is called vertex-optimal if for each vertex $v \in C$ and $C' \in \mathbb{C}$, the modularity gain $\Delta Q(v \rightarrow C', \gamma) \leq 0$.*

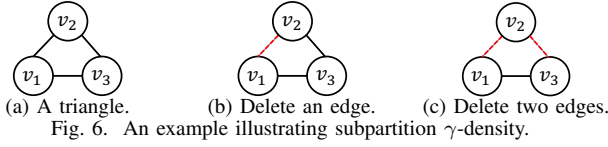
Next, we introduce an assumption in the maintenance of Louvain communities [10], [11]:

Assumption 1. *The sum of weights of the updated edges is sufficiently small relative to the graph size m .*

Based on Assumption 1, prior studies suggest that when the number of edge updates is small relative to the graph size, three heuristics hold: (1) intra-community edge deletions and inter-community edge insertions could affect vertex-level community membership [10], [11]; (2) Inter-community edge deletions and intra-community edge insertions can be ignored [10], [11]; (3) Vertices directly involved in such edge changes are the most likely to alter their communities [10]. The heuristics are stated in Observation 1, which can be proved based on Definition 6.

Observation 1 ([10]). *Given an intra-community edge deletion $(v_i, v_j, -\alpha)$ or a cross-community edge insertion (v_i, v_j, α) , its effect on the community memberships of vertices v_i and v_j cannot be ignored.*

We further derive the propagation of community changes from Observation 1.



Lemma 1. *When a vertex v changes its community to C , then the communities of its neighbors not in C in the updated graph could be affected.*

Based on these analyses, we develop a novel movement phase, called `inc-movement` in HIT-Leiden to preserve vertex optimality, which will be introduced in Section V-A.

• **Analysis of subpartition γ -density.** For simplified analysis, we first introduce γ -order and γ -connectivity, which are key concepts for defining subpartition γ -density.

Definition 7 (γ -order). *Given two vertex sequences X and Y of a graph G , let $X \otimes Y$ represent that Y is merged into X such that $2m \cdot w(X, Y) \geq \gamma \cdot d(X) \cdot d(Y)$, where $w(X, Y) = \sum_{v_i \in X} \sum_{v_j \in Y} w(v_i, v_j)$. A γ -order of a vertex sequence $U = \{v_1, \dots, v_x\}$ represents the merged sequence starting from singleton sequences $\{v_1\}, \dots, \{v_x\}$.*

We can maintain one γ -order per sub-community from Leiden, which is represented by the sequence of vertices merging into the sub-community in refinement phase of Leiden.

Definition 8 (γ -connectivity [5]). *Given a graph G , a vertex sequence U is γ -connected if U can be generated from at least one γ -order.*

Definition 9 (Subpartition γ -density [5]). *A vertex subsequence $U \subseteq C \in \mathbb{C}$ is subpartition γ -dense if U is γ -connected, and any intermediate vertex sequence X is locally optimized, i.e., $\Delta Q(X \rightarrow \emptyset, \gamma) \leq 0$.*

Notably, $\Delta Q(X \rightarrow \emptyset, \gamma) \leq 0$ denotes the modularity gain of moving X from C to an empty set, whose calculation follows the same formula as the standard modularity gain in (2).

Example 2. *The triangle in Fig. 6(a) is subpartition γ -dense with $\gamma = 1$ since there are six different γ -orders. For instance, one is $\{v_3\} \otimes (\{v_1\} \otimes \{v_2\})$, which represents that v_2 is merged into $\{v_1\}$ generating sequence $\{v_1, v_2\}$, and then $\{v_1, v_2\}$ is merged into v_3 generating $\{v_1, v_2, v_3\}$. After deleting the edge (v_1, v_2) , although $\{v_3\} \otimes (\{v_1\} \otimes \{v_2\})$ is not a γ -order, the updated graph is still subpartition γ -dense since $\{v_1\} \otimes (\{v_2\} \otimes \{v_3\})$ is a γ -order in the updated graph. After continuing to delete the edge (v_2, v_3) , the updated graph is not subpartition γ -dense since v_2 is not connected to v_1 and v_3 .*

In essence, each community C (or sub-community S) of Leiden is subpartition γ -dense, since (1) any sub-community in C (or S) is locally optimized, and (2) all sub-communities are γ -connected. Notably, as shown in Fig. 3(b), vertex optimality ensures the first condition by design since any sub-community will be a supervertex in `inc-movement` of the next iteration. Thus, we will develop a new refinement

algorithm, `inc-refinement`, to preserve γ -connectivity of sub-communities.

Next, we analyze the γ -connectivity property under two kinds of graph updates, i.e., *edge deletion* and *edge insertion*. For any vertex v_i within a sub-community S_i with a γ -order, we denote an intermediate subsequence of the γ -order containing v_i by $I_i \subseteq S_i$, and the subsequence $U_i = I_i \setminus \{v_i\}$ is an intermediate subsequence of the γ -order before merging v_i .

(1) *Edge deletion.* We consider the deletions of both intra-sub-community edges and cross-sub-community edges:

Lemma 2. *Given an intra-sub-community edge deletion $(v_i, v_j, -\alpha)$, assume v_j is before v_i in the γ -order of the sub-community. The effects of the edge deletion can be described by the following four cases:*

- 1) v_i could be removed from its sub-community only if $\alpha > \frac{2m \cdot w(v_i, U_i) - \gamma \cdot d(v_i) \cdot d(U_i)}{4m + 2w(v_i, U_i)}$;
- 2) v_j could be removed from its sub-community only if $\alpha > m - \frac{\gamma \cdot d(v_j) \cdot d(U_j)}{2w(v_j, U_j)}$;
- 3) For any $v_k \in S_i$ ($k \neq i, j$), it could be removed from its sub-community only if $\alpha > m - \frac{\gamma \cdot d(v_k) \cdot d(U_k)}{2w(v_k, U_k)}$;
- 4) For any $v_l \notin S_i$, it should be removed from its sub-community if and only if $\alpha > m - \frac{\gamma \cdot d(v_l) \cdot d(U_l)}{2w(v_l, U_l)}$.

Lemma 3. *Given a cross-sub-community edge deletion $(v_i, v_j, -\alpha)$, the effects of the edge deletion can be described by the following four cases:*

- 1) v_i could be removed from its sub-community only if $\alpha > m - \frac{\gamma \cdot d(v_i) \cdot d(U_i)}{2w(v_i, U_i)}$;
- 2) v_j holds similar behavior with v_i ;
- 3) For any $v_k \in S_i \cup S_j$ ($k \neq i, j$), it could be removed from its sub-community only if $\alpha > m - \frac{\gamma \cdot d(v_k) \cdot d(U_k)}{2w(v_k, U_k)}$;
- 4) For any $v_l \notin S_i \cup S_j$, it could be removed from its sub-community only if $\alpha > m - \frac{\gamma \cdot d(v_l) \cdot d(U_l)}{2w(v_l, U_l)}$.

(2) *Edge insertion.* We consider the insertion of edges containing the insertions of both intra-sub-community edges and cross-sub-community edges:

Lemma 4. *Given an edge insertion (v_i, v_j, α) , the effects of the edge insertion can be described by the following four cases:*

- 1) v_i could be removed from its sub-community only if $\alpha > \frac{4}{\gamma}m - d(I_i)$ or $\alpha > \frac{2w(v_i, U_i)}{\gamma \cdot d(U_i)} \cdot m - d(v_i)$;
- 2) v_j could be removed from its sub-community, only if $\alpha > \frac{2w(v_j, U_j)}{\gamma \cdot d(U_j)} \cdot m - d(v_j)$;
- 3) For any $v_k \in S_i \cup S_j$ ($k \neq i, j$), it could be removed from its sub-community, only if $\alpha > \frac{w(v_k, U_k)}{\gamma \cdot d(v_k)} \cdot m - \frac{1}{2}d(U_k)$;
- 4) For any $v_l \notin S_i \cup S_j$, it is unaffected.

Observation 2. *In the refinement phase of Leiden algorithms, each vertex v tends to be merged into the sub-community (intermediate subsequence U), offering more edge weights $w(v, U)$ and smaller degrees $d(U)$. Therefore, the differences in the values of $d(v)$, $w(v, U)$, and $d(U)$ are typically very small.*

Therefore, α is unlikely to satisfy the conditions in cases (2)-(4) of Lemma 2, all the cases of Lemma 3, and the conditions in cases (1)-(3) of Lemma 4 when $\alpha \ll m$ (which is often true as stated in Assumption 1). As a result, when designing the maintenance algorithm, we only need to consider the effect of intra-sub-community edge deletions on v_i , which cannot be ignored.

Besides, our experiments show the following observation, which shows that the case (1) of Lemma 2 can also be ignored.

Observation 3. *Given an updated graph with its previous sub-community memberships, for any sub-community S , we treat each connected component in S as a new sub-community. Most of the maintained communities are subpartition γ -dense.*

The above observation holds because Leiden only offers us a γ -order from the refinement phase, and a subgraph often exists with multiple distinct γ -orders as shown in Example 2. Besides, if a vertex is a candidate affecting γ -connectivity, it is often a candidate affecting vertex optimality, e.g., the vertex v_2 in Fig. 6(c). In this case, the vertex is likely to change its community before verifying whether the vertex needs to move out of its sub-community. Hence, the case (1) of Lemma 2 can be ignored if the intra-sub-community edge deletion does not cause the sub-community to be disconnected.

Based on Observations 2-3, we develop a novel refinement algorithm, called *inc-refinement*, in HIT-Leiden, which will be introduced in Section V-B.

As shown in Fig. 13, over 99% maintained communities from HIT-Leiden are subpartition γ -dense.

Extension to supergraphs. Changes at the lower level propagate upward to superedge changes in the higher-level supergraph, as Leiden constructs a list of supergraphs in a bottom-up manner. This motivates us to develop an incremental aggregation phase, namely *inc-aggregation*, to compute the superedge changes in Section V-C.

Example 3. In Fig. 1, communities C_1 and C_2 are treated as supervertices. Deleting an edge $(v_3, v_5, 1)$ and inserting an edge $(v_1, v_3, 1)$ cause v_3 and v_4 to move from C_2 to C_1 . This results in the deletion of $(C_2, C_2, -2)$ and insertion of $(C_1, C_1, 2)$ in the supergraph.

Therefore, we treat each supergraph as a set of facing edge changes from the previous Leiden community and process them using a consistent procedure as shown in Fig. 3(b).

Characterization of AFF. Based on these analyses, we define the supervertices that change their communities or sub-communities as the affected area AFF of Leiden.

V. OUR HIT-LEIDEN ALGORITHM

In this section, we propose HIT-Leiden, which updates community memberships by restricting computation to the affected region, as motivated by the analysis in Section IV-B. We first introduce its three key components, namely *inc-movement*, *inc-refinement*, and *inc-aggregation* of our HIT-Leiden. Fig. 7 shows the assumption, lemmas, and observations used in these components. Then, we present an auxiliary procedure, called

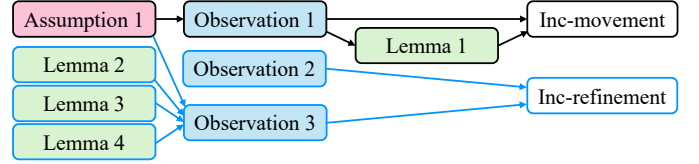
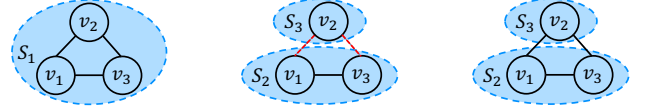


Fig. 7. The design rationale for *inc-movement* and *inc-refinement*.



(a) A sub-community. (b) Delete two edges. (c) Move out a vertex. Fig. 8. Illustrating the process that a sub-community S_1 is split into two sub-communities S_2 and S_3 .

deferred update, abbreviated as *def-update*. Afterward, we give an overview of HIT-Leiden, and finally analyze the boundedness of HIT-Leiden.

A. Inc-movement

The goal of *inc-movement* is to preserve vertex optimality. As analyzed in Section IV-B, the endpoints of a deleted intra-community edge or an inserted cross-community edge may affect their community memberships. If an affected vertex changes its community, its neighbors outside the target community may also be affected. Note that any vertex that changes its community has to change its sub-community, since each sub-community is a subset of its community. Hence, sub-community memberships are also considered in *inc-movement*.

We first introduce the data structures used to maintain a dynamic sub-community. According to Observation 3, each connected component of a sub-community is treated as a γ -connected subset. When edge updates or vertex movements split a sub-community into multiple connected components, we re-assign each resulting component as a new sub-community.

Example 4. Fig. 8 shows the sub-community S_1 is split into two sub-communities $S_2 = \{v_1, v_3\}$ and $S_3 = \{v_2\}$. The separation can occur either due to the deletion of edges (v_1, v_2) and (v_2, v_3) during graph updates, as shown in Fig. 8(b), or due to the removal of vertex v_2 during the movement phase, as shown in Fig. 8(c).

To preserve the structure under such changes, we leverage the dynamic connected component maintenance technique in [15]. The graph G_Ψ stores the subgraph of G consisting only of intra-sub-community edges based on $s(\cdot)$.

Algorithm 2 shows *inc-movement*. Given an updated graph G , a set of graph changes ΔG , community mappings $f(\cdot)$, sub-community mappings $s(\cdot)$, and a CC-index Ψ , it first initializes three empty sets: A , B and K (line 1). Here, A keeps the vertices whose community memberships may be changed, B keeps the vertices that have changed their community memberships, and K records the endpoints on edges whose deletion disconnects the connected component in G_Ψ . Subsequently, vertices involved in intra-community edge deletion or cross-community edge insertion are added to A , and edges in G_Ψ are updated according to intra-sub-community changes (lines 2-7) based on Observations 1 and

Algorithm 2: Inc-movement

Input: $G, \Delta G, f(\cdot), s(\cdot), \Psi, \gamma$
Output: Updated $f(\cdot), \Psi, B, K$

```

1  $A \leftarrow \emptyset, B \leftarrow \emptyset, K \leftarrow \emptyset;$ 
2 for  $(v_i, v_j, \alpha) \in \Delta G$  do
3   if  $\alpha > 0$  and  $f(v_i) \neq f(v_j)$  then
4      $A.add(v_i); A.add(v_j);$ 
5   if  $\alpha < 0$  and  $f(v_i) = f(v_j)$  then
6      $A.add(v_i); A.add(v_j);$ 
7   if  $s(v_i) = s(v_j)$  and  $update\_edge(G_\Psi, (v_i, v_j, \alpha))$  then
8      $K.add(v_i); K.add(v_j);$ 
9 while  $A \neq \emptyset$  do
10    $v_i \leftarrow A.pop();$ 
11    $C^* \leftarrow \operatorname{argmax}_{C \in \mathcal{C} \cup \{\emptyset\}} \Delta Q(v_i \rightarrow C, \gamma);$ 
12   if  $\Delta Q(v_i \rightarrow C^*, \gamma) > 0$  then
13      $f(v_i) \leftarrow C^*; B.add(v_i); K.add(v_i);$ 
14     for  $v_j \in N(v_i)$  do
15       if  $f(v_j) \neq C^*$  then
16          $A.add(v_j);$ 
17     for  $v_j \in \{u \in N(v_i) | s(u) = s(v_i)\}$  do
18       if  $update\_edge(G_\Psi, (v_i, v_j, -w(v_i, v_j)))$  then
19          $K.add(v_j);$ 
20 return  $f(\cdot), \Psi, B, K;$ 

```

3, respectively. If an edge update in G_Ψ causes a connected component to split (i.e., $update_edge(\cdot)$ returns *true*), its endpoints are added to K (line 8). It then processes vertices in A until the set is empty (line 9). For each vertex v_i , it identifies the target community C^* that yields the highest modularity gain (lines 10-11). If $\Delta Q(v_i \rightarrow C^*) > 0$, $f(v_i)$ is updated to C^* , v_i is added to B and K , and the neighbors of v_i not in C^* are added to A (lines 12-16), which implements the property in Lemma 1. Besides, the intra-sub-community edges involving v_i are deleted from G_Ψ , and the vertices involved in component splits are added to K (lines 17-19). Finally, it returns $f(\cdot), \Psi, B$, and K (line 20).

B. Inc-refinement

As discussed in Section V-A and Observation 3, we treat each connected component in G_Ψ maintained in *inc-movement* as a sub-community. Therefore, we design *inc-refinement* for re-assigning each new connected component in G_Ψ as a sub-community. Additionally, we attempt to merge singleton sub-communities whose process is the same as the process of the refinement phase in Leiden with G_Ψ maintenance.

Algorithm 3 presents its pseudocode. Given an updated graph G , community mappings $f(\cdot)$ and sub-community mapping $s(\cdot)$, a CC-index Ψ , and a set K , it first initializes R as an empty list to track vertices that have changed their sub-communities (line 1). It then traverses K to identify split connected components in G_Ψ using breadth-first search or depth-first search. All vertices in the split connected component are re-mapped to a new sub-community and added to R (lines 2-3). For each vertex $v_i \in R$ that is in a singleton sub-community, *inc-refinement* uses a set $\mathcal{T}$ to store the

Algorithm 3: Inc-refinement

Input: $G, f(\cdot), s(\cdot), \Psi, K, \gamma$
Output: Updated $s(\cdot), \Psi, R$

```

1  $R \leftarrow \emptyset;$ 
2 for  $v_i \in K$  do
3   Map all vertices in the connected component containing
    $v_i$  into a new sub-community and add them into  $R;$ 
4 for  $v_i \in R$  do
5   if  $v_i$  is in a singleton sub-community then
6      $\mathcal{T} \leftarrow \{s(v) | v \in N(v_i), f(v) = f(v_i), \Delta Q(s(v) \rightarrow \emptyset, \gamma) \leq 0\};$ 
7     if  $\mathcal{T} = \emptyset$  then
8       continue;
9      $S^* \leftarrow \operatorname{argmax}_{S \in \mathcal{T}} \Delta Q(v_i \rightarrow S, \gamma);$ 
10    if  $\Delta Q(v_i \rightarrow S^*, \gamma) > 0$  then
11       $s(v_i) \leftarrow S^*;$ 
12      for  $v_j \in N(v_i)$  do
13        if  $s(v_i) = s(v_j)$  then
14           $update\_edge(G_\Psi, (v_i, v_j, w(v_i, v_j)));$ 
15 return  $s(\cdot), \Psi, R;$ 

```

locally optimized neighboring sub-communities of v_i within the same community (lines 4-6). If $\mathcal{T} \neq \emptyset$, it attempts to re-assign v_i to a sub-community $S^* \in \mathcal{T}$, which offers the highest modularity gain to eliminate singleton sub-communities (lines 7-9). Notably, $\Delta Q(v_i \rightarrow S, \gamma)$ denotes the modularity gain of moving v_i from $s(v_i)$ to S , whose calculation follows the same formula as the standard modularity gain. If the gain is positive, $s(v_i)$ is updated to S^* , and the corresponding intra-sub-community edges are added into G_Ψ (lines 10-14). Finally, *inc-refinement* returns the $s(\cdot), \Psi$, and R (line 15).

C. Inc-aggregation

Given an updated graph G and its edge changes ΔG , modifications to edges and sub-community memberships are reflected as changes to superedges and supervertices in the supergraph H . Let $s_{pre}(\cdot)$ (resp. $s_{cur}(\cdot)$) denote the vertex-to-supervertex mappings before (resp. after) *inc-refinement*. Any edge change (v_i, v_j, α) in ΔG corresponds to a superedge change $(s_{pre}(v_i), s_{pre}(v_j), \alpha)$ in H , since the weight of a superedge is the sum of the weights of edges between their sub-communities. Besides, a vertex v migration from $s_{pre}(v)$ to $s_{cur}(v)$ requires updating these weights. Specifically, the original sub-community $s_{pre}(v)$ must decrease the superedge weights corresponding to the edge incident to v , and the new sub-community $s_{cur}(v)$ must increase them under the new assignment.

Example 5. Following Example 3, the initial superedge changes due to edge changes are $(C_1, C_2, 1)$ and $(C_2, C_2, -1)$. Then, vertices v_3 and v_4 move from C_2 to C_1 , and there are three cases:

- 1) C_1 gains edges to the neighbors of v_3 , resulting in two updates: $(C_1, C_1, 1)$ and $(C_1, C_1, 1)$;
- 2) C_2 loses edges incident to the neighbors of v_3 , resulting in $(C_1, C_2, -1)$ and $(C_2, C_2, -1)$;
- 3) The effect of v_4 is skipped to avoid duplicate updates, since its only neighbor v_3 already changed.

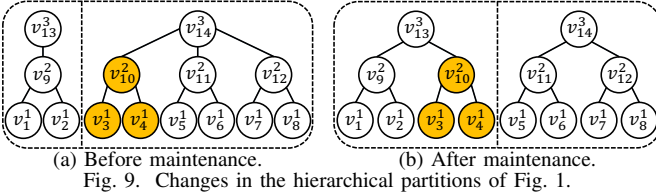
Algorithm 4: Inc-aggregation

Input: $G, \Delta G, s_{pre}(\cdot), s_{cur}(\cdot), R$
Output: $\Delta H, s_{pre}(\cdot)$

```

1  $\Delta H \leftarrow \emptyset;$ 
2 for  $(v_i, v_j, \alpha) \in \Delta G$  do
3    $s_i \leftarrow s_{pre}(v_i), s_j \leftarrow s_{pre}(v_j);$ 
4    $\Delta H.add((s_i, s_j, \alpha));$ 
5 for  $v_i \in R$  do
6   for  $v_j \in N(v_i)$  do
7     if  $s_{cur}(v_j) = s_{pre}(v_j)$  or  $i < j$  then
8        $\Delta H.add((s_{pre}(v_i), s_{pre}(v_j), -w(v_i, v_j)));$ 
9        $\Delta H.add((s_{cur}(v_i), s_{cur}(v_j), w(v_i, v_j)));$ 
10     $\Delta H.add((s_{pre}(v_i), s_{pre}(v_i), -w(v_i, v_i)));$ 
11     $\Delta H.add((s_{cur}(v_i), s_{cur}(v_i), w(v_i, v_i)));$ 
12 for  $v_i \in R$  do
13    $s_{pre}(v_i) \leftarrow s_{cur}(v_i);$ 
14  $Compress(\Delta H);$ 
15 return  $\Delta H, s_{pre}(\cdot);$ 

```



After compressing the above six superedge changes, we obtain the final superedge changes, which are $(C_1, C_1, 2)$ and $(C_2, C_2, -2)$.

Algorithm 4 presents inc-aggregation. Initially, the set of changes ΔH of H is empty (line 1). Then, it maps the edge changes ΔG to superedge changes using $s_{pre}(\cdot)$ (lines 2-4). Following, it updates superedges for vertices that switch sub-communities by removing edges from the old community and adding edges to the new one. For any vertex v_i in R , it updates superedges with each neighbor v_j if either $s_{cur}(v_j) = s_{pre}(v_j)$ or $i < j$ to avoid duplicate processing (lines 5-9). Besides, it updates the self-loop for the sub-community of v_i (lines 10-11). Finally, it updates $s_{pre}(\cdot)$ locally to match $s_{cur}(\cdot)$ at the next time step (lines 12-13) and compresses ΔH by summing the weights of identical superedges (line 14).

D. Overall HIT-Leiden algorithm

Before presenting our overall HIT-Leiden algorithm, we introduce an optimization technique to further improve the efficiency of the vertices' membership update. Specifically, when a supervertex changes its community membership, all the lower-level supervertices associated with it have to update their community membership. As shown in Fig. 9, when v_{10}^2 changes its community, v_3^1 and v_4^1 also update their community memberships to the community containing v_{10}^2 . However, during the iteration process of HIT-Leiden, a supervertex that changes its community does not automatically trigger updates of the community memberships of its constituent lower-level supervertices.

To resolve the above inconsistency, we perform a post-processing step to synchronize the community memberships across all levels, as described in Algorithm 5. Let $\{B^P\}$

Algorithm 5: def-update

Input: $\{f^P(\cdot)\}, \{s^P(\cdot)\}, \{B^P\}, P$
Output: Updated $\{f^P(\cdot)\}$

```

1 for  $p$  from  $P$  to 1 do
2   if  $p \neq P$  then
3     for  $v_i^p \in B^p$  do
4        $f^p(v_i^p) \leftarrow f^{p+1}(s^{p+1}(v_i^p));$ 
5   if  $p \neq 1$  then
6     for  $v_i^p \in B^p$  do
7        $B^{p-1}.add((s^{p-1})^{-1}(v_i^p));$ 
8 return  $\{f^P(\cdot)\};$ 

```

denote a sequence of P sets $\{B^1, \dots, B^P\}$, $\{s^P(\cdot)\}$ denote a sequence of P adjacent-level supervertex mappings $\{s^1(\cdot), \dots, s^P(\cdot)\}$, and $\{f^P(\cdot)\}$ denote a sequence of P community mappings $\{f^1(\cdot), \dots, f^P(\cdot)\}$. Note, each B^p in $\{B^P\}$ collects supervertices at level- p whose community memberships have changed, each $s^p(\cdot)$ in $\{s^P(\cdot)\}$ maps from level- p supervertices to their parent supervertices at level- $(p+1)$, and each $f^p(\cdot)$ in $\{f^P(\cdot)\}$ maps from level- p supervertices to their communities. A supervertex is added to B^p for one of two reasons: (1) it changes its community during inc-movement, or (2) its higher-level supervertex changes community. Hence, for each level p , def-update updates each supervertex in B^p by re-mapping its community membership of its parent using $s^p(\cdot)$ and $f^{p+1}(\cdot)$ when $p \neq P$ (lines 1-4), and adds its constituent vertices to B^{p-1} for the next level updates where $(s^{p-1})^{-1}(\cdot)$ denotes the inverse mapping of $s^{p-1}(\cdot)$ (lines 5-7). This algorithm also supports updating the ancestor mappings $\{g^P(\cdot)\}$ from each level supervertex to its level- P ancestor.

• **Overall HIT-Leiden.** After introducing all the key components, we present our overall HIT-Leiden in Algorithm 6. The algorithm proceeds over P hierarchical levels, where each level- p operates on a corresponding supergraph G^p . Besides the community membership $f(\cdot)$, HIT-Leiden also maintains supergraphs $\{G^P\}$, community mapping $\{f^P(\cdot)\}$, ancestor mappings $\{g^P(\cdot)\}$, sub-community mappings $\{s_{pre}^P(\cdot)\}$ and $\{s_{cur}^P(\cdot)\}$, and CC-indices $\{\Psi^P\}$ to maintain sub-community memberships for each level. Note, $\{s_{pre}^P(\cdot)\}$ are the mappings from the previous time step, and $\{s_{cur}^P(\cdot)\}$ are the in-time mappings to track sub-community memberships as they evolve at the current time step. Specifically, it initializes $\{s_{cur}^P(\cdot)\} = \{s_{pre}^P(\cdot)\}$. Given the graph change ΔG , it first initializes the first-level update ΔG to ΔG^1 (line 1). It then proceeds through P iterations, each including three phases after updating the supergraph G^p (line 3).

- 1) Inc-movement (line 4): it re-assigns community memberships of affected vertices to achieve vertex optimality, which yields $f^p(\cdot)$, Ψ , B^p , and K .
- 2) Inc-refinement (line 5): it re-maps the supervertices of split connected components in Ψ to new sub-communities, producing $s_{cur}^p(\cdot)$, Ψ , and R^p .
- 3) Inc-aggregation (line 7): it calculates the next level's superedge changes ΔG^{p+1} , and synchronizes $s_{pre}^p(\cdot)$ to match $s_{cur}^p(\cdot)$.

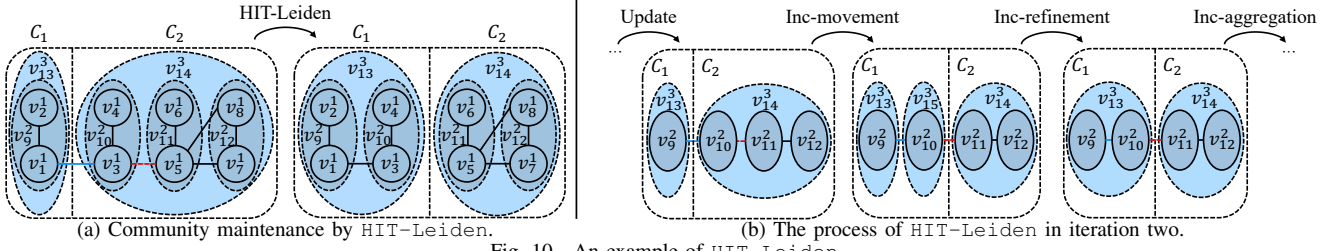


Fig. 10. An example of HIT-Leiden.

Algorithm 6: HIT-Leiden

Input: $\{G^P\}$, ΔG , $\{f^P(\cdot)\}$, $\{g^P(\cdot)\}$, $\{s_{pre}^P(\cdot)\}$, $\{s_{cur}^P(\cdot)\}$, $\{\Psi^P\}$, P , γ
Output: $f(\cdot)$, $\{G^P\}$, $\{f^P(\cdot)\}$, $\{g^P(\cdot)\}$, $\{s_{pre}^P(\cdot)\}$, $\{s_{cur}^P(\cdot)\}$, $\{\Psi^P\}$

```

1  $\Delta G^1 \leftarrow \Delta G$ ;
2 for  $p = 1$  to  $P$  do
3    $G^p \leftarrow G^p \oplus \Delta G^p$ ;
4    $f^p(\cdot), \Psi^p, B^p, K \leftarrow$ 
     inc-movement( $G^p, \Delta G^p, f^p(\cdot), s_{cur}^p(\cdot), \Psi^p, \gamma$ );
5    $s_{cur}^p(\cdot), \Psi^p, R^p \leftarrow$ 
     inc-refinement( $G^p, f^p(\cdot), s_{cur}^p(\cdot), \Psi^p, K, \gamma$ );
6   if  $p < P$  then
7      $\Delta G^{p+1}, s_{pre}^{p+1}(\cdot) \leftarrow$ 
       inc-aggregation( $G^p, \Delta G^p, s_{pre}^p(\cdot), s_{cur}^p(\cdot), R^p$ );
8  $\{f^P(\cdot)\} \leftarrow$  def-update( $\{f^P(\cdot)\}, \{s_{cur}^P(\cdot)\}, \{B^P\}, P$ );
9  $\{g^P(\cdot)\} \leftarrow$  def-update( $\{g^P(\cdot)\}, \{s_{cur}^P(\cdot)\}, \{R^P\}, P$ );
10  $f(\cdot) \leftarrow g^1(\cdot)$ ;
11 return  $f(\cdot), \{G^P\}, \{f^P(\cdot)\}, \{g^P(\cdot)\}, \{s_{pre}^P(\cdot)\}, \{s_{cur}^P(\cdot)\}, \{\Psi^P\}$ ;
```

After P iterations, def-update (Algorithm 5) synchronizes community mappings $\{f^P(\cdot)\}$ and ancestor mappings $\{g^P(\cdot)\}$ across levels (lines 8-9). The final output $f(\cdot)$ is set to $g^1(\cdot)$ (line 10), since $f^1(\cdot)$ could contain disconnected communities with insufficient iterations and $g^1(\cdot) = f^1(\cdot)$ with sufficient iterations. Finally, we return $\{G^P\}$, $\{f^P(\cdot)\}$, $\{g^P(\cdot)\}$, $\{s_{pre}^P(\cdot)\}$, $\{s_{cur}^P(\cdot)\}$, and $\{\Psi^P\}$ for the next graph evolution (line 11).

Example 6. Consider the result in Fig. 4. The graph undergoes an edge deletion $(v_3^1, v_5^1, -1)$ and an edge insertion $(v_1^1, v_3^1, 1)$. Resulting community and sub-community changes are shown in Fig. 10, with hierarchical changes in Fig. 9. Take the second iteration as an example. In inc-movement, the supervertex v_{10}^2 is reassigned to v_{15}^3 due to disconnection, and migrates from community C_2 to C_1 . In inc-refinement, v_{10}^2 is merged into v_{13}^3 . Then, inc-aggregation calculates superedge changes for level-3, including edge insertion $(v_{13}^3, v_{13}^3, 2)$ and edge deletions $(v_{14}^3, v_{14}^3, -2)$.

• **Complexity analysis.** We now analyze the time and space complexity of HIT-Leiden over P iterations. Let Γ^p denote the set of supervertices involved in superedge changes, let Λ^p track the supervertices that change their communities or sub-communities at level- p , and let h^p denote the average vertex depth in the spanning tree in [15] at level- p . Typically, h^p is treated as a small factor. For each level- p , inc-movement and inc-refinement com-

TABLE III
DATASETS USED IN OUR EXPERIMENTS.

Dataset	Abbr.	$ V $	$ E $	Timestamp
dblp-coauthor	DC	1.8M	29.4M	Yes
yahoo-song	YS	1.6M	256.8M	Yes
sx-stackoverflow	SS	2.6M	63.4M	Yes
it-2004	IT	41.2M	1.0B	No
risk	RS	201.0M	4.0B	Yes

plete in $O(|N_2(\Gamma^p)| + h^p \cdot |\Gamma^p| + |N_2(\Lambda^p)| + h^p \cdot |\Lambda^p|)$, and inc-aggregation completes in $O(|N(\Gamma^p)| + |N(\Lambda^p)|)$. Besides, the time cost of def-update is $O(\sum_{p=1}^P |\Lambda^p|)$. Hence, the total time cost of HIT-Leiden can be expressed as $O(|N_2(\text{CHANGED})| + |N_2(\text{AFF})|)$, as analyzed in Section IV-B. As a result, our HIT-Leiden is relatively bounded to Leiden. In the worst case, HIT-Leiden degenerates to Leiden with a full rebuild of the CC-index at each level. The total time is upper-bounded by $O(\sum_{p=1}^P (|V^p| + (1 + h^p) \cdot |E^p|))$. For space complexity, at each level- p , HIT-Leiden stores the aggregated graph, community/sub-community assignments, auxiliary update sets, and the CC-index. Since the CC-index requires linear space with respect to the number of vertices and edges, the space cost at level- p is $O(|V^p| + |E^p|)$. Thus, the total space cost is at most $O(\sum_{p=1}^P (|V^p| + |E^p|))$.

VI. EXPERIMENTS

We now present our experimental results. Section VI-A introduces the experimental setup. Sections VI-B and VI-C evaluate the effectiveness and efficiency of HIT-Leiden, respectively.

A. Setup

Datasets. We use four real-world dynamic datasets, including *dblp-coauthor*² (academic collaboration), *yahoo-song*² (user-song interactions), *sx-stackoverflow*³ (developer Q&A), and *risk* (financial transactions) provided by ByteDance. All these dynamic edges are associated with real timestamps. We also use one static dataset *it-2004*⁴ (a large-scale web graph), but randomly insert or delete some edges to simulate a dynamic graph. All the graphs are treated as undirected graphs. For each real-world dynamic graph, we collect a sequence of batch updates by sorting the edges in ascending order of their timestamps; for *it-2004*, which lacks timestamps, we randomly shuffle its edge order. Table III summarizes the key statistics of the above datasets.

²<http://konect.cc/networks/>

³<https://snap.stanford.edu/data/>

⁴<https://networkrepository.com/>

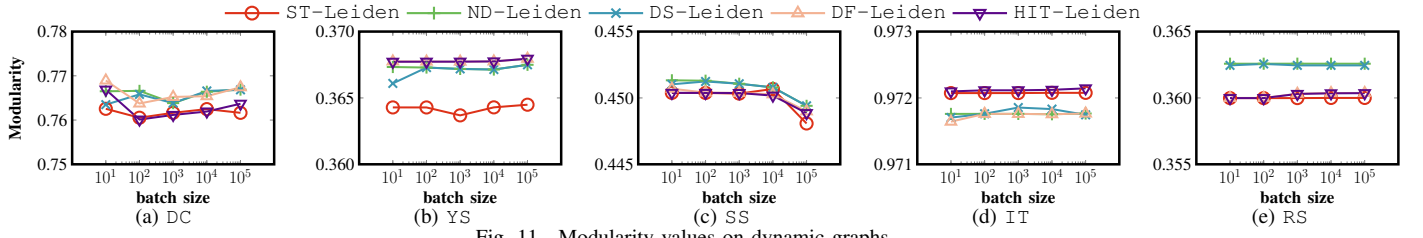


Fig. 11. Modularity values on dynamic graphs.

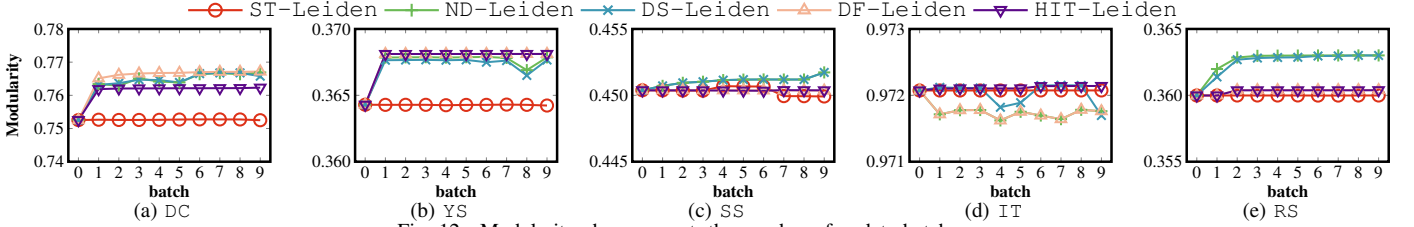


Fig. 12. Modularity changes w.r.t. the number of update batches.

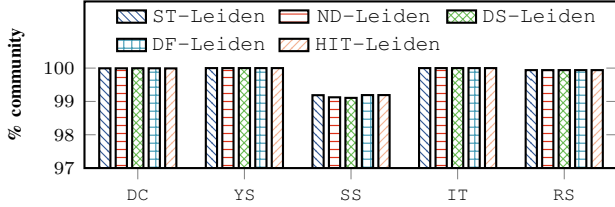


Fig. 13. Percentage of subpartition γ -dense communities.

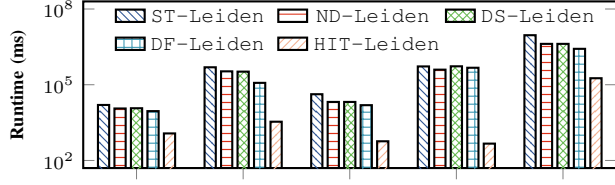


Fig. 14. Efficiency of all Leiden algorithms on all datasets.

Algorithms. We test the following maintenance algorithms:

- **ST-Leiden:** A naive baseline that executes the static Leiden algorithm from scratch when the graph changes.
- **ND-Leiden:** A simple maintenance algorithm in [14], which processes all vertices during the movement phase, initialized with previous community memberships.
- **DS-Leiden:** A maintenance algorithm based on [14], which uses the delta-screening technique [11] to restrict the number of vertices considered in the movement phase.
- **DF-Leiden:** An advanced maintenance algorithm from [14], which adopts the dynamic frontier approach [10] to support localized updates.
- **HIT-Leiden:** Our proposed method.

Dynamic graph settings. As the temporal span varies across datasets (e.g., 62 years for *dblp-coauthor* versus 8 years for *sx-stackoverflow*), we apply a sliding edge window, avoiding reliance on fixed valid time intervals that are hard to standardize. Initially, we construct a static graph using the first 80% of edges. Then, we select a window size $b \in \{10, 10^2, 10^3, 10^4, 10^5\}$, denoting the number of updated edges in an updated batch. Next, we slide this window $r = 9$ times, so we update 9 batches of edges for each dataset. In the figures, batch 0 denotes the initial state before the first evaluated update. Note that by default, we set $b = 10^3$.

All the algorithms are implemented in C++ and compiled

with the gcc 8.3.0 compiler using the same optimization level. We set $\gamma = 1$ and use $P = 10$ iterations. Before running the Leiden community maintenance algorithms, we obtain the communities by running the Leiden algorithm, and HIT-Leiden requires an additional procedure to build auxiliary structures. Due to the limited number of iterations, the community structure has not fully converged, so the maintenance algorithms usually take more time in the first two batches than in other batches. Therefore, we exclude the first two batches from efficiency evaluations. All experiments in this paper are conducted only on a Linux server running Debian 5.4.56, equipped with an Intel(R) Xeon(R) Platinum 8336C CPU @ 2.30GHz and 2.0 TB of RAM.

B. Effectiveness evaluation

To evaluate the effectiveness of different maintenance algorithms, we compare the modularity value and proportion of subpartition γ -dense communities for their returned communities.

• **Modularity.** Fig. 11 depicts the average modularity values of all the maintenance algorithms, where the batch size ranges from 10 to 10^5 . Fig. 12 depicts the modularity value across all 9 update batches, where the batch size is fixed as 1,000. Across all datasets, the expected fluctuation in modularity for ST-Leiden is around 0.02 due to its inherent randomness. These maintenance algorithms achieve equivalent quality in modularity, since the difference in their modularity values is within 0.01. Overall, our HIT-Leiden achieves comparable modularity with other methods.

• **Proportion of subpartition γ -density.** After running HIT-Leiden, for each returned community, we try to re-find its γ -order such that any intermediate vertex set in the γ -order is locally optimized, according to Definition 9. If we can find a valid γ -order for a community, we classify it as a subpartition γ -dense community. We report the proportion of subpartition γ -dense communities in Fig. 13. The proportions of subpartition γ -dense communities are consistently high across all Leiden-based algorithms. For each dataset, the differences among these algorithms are very small, with fluctuations around 0.0001. Thus, HIT-Leiden achieves a comparable percentage of subpartition γ -density with others.

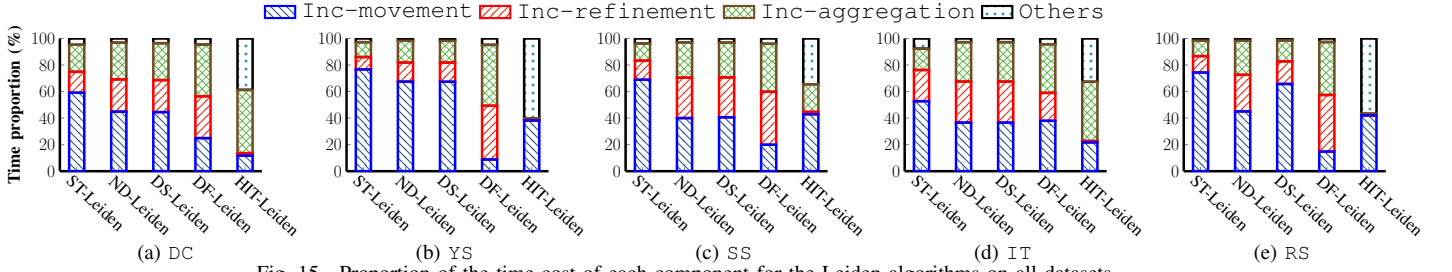


Fig. 15. Proportion of the time cost of each component for the Leiden algorithms on all datasets.

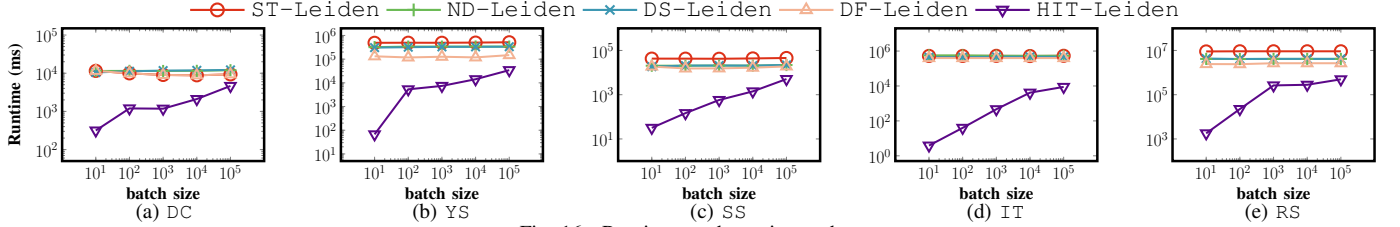


Fig. 16. Runtime on dynamic graphs.

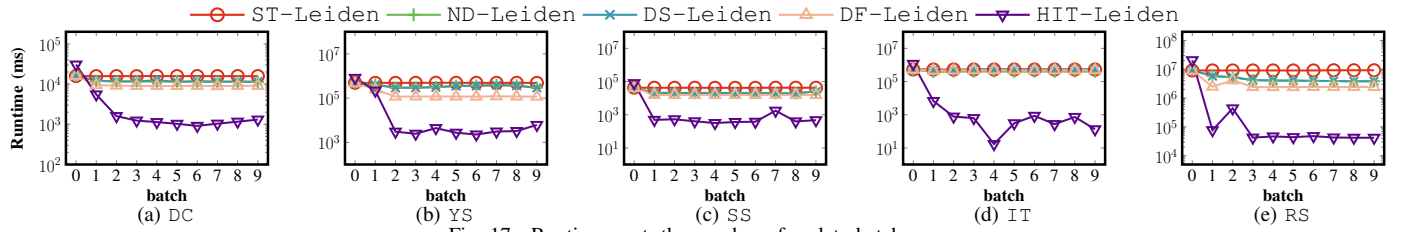


Fig. 17. Runtime w.r.t. the number of update batches.

C. Efficiency evaluation

In this section, we first present the overall efficiency results, then analyze the time cost of each component, and finally evaluate the effects of some hyperparameters.

• **Overall results.** Fig. 14 presents the overall efficiency results where b is set to its default value 1,000. Clearly, HIT-Leiden achieves the best efficiency on datasets, especially on the *it-2004* dataset, since it is up to three orders of magnitude faster than the state-of-the-art algorithms. That is mainly because the ratio of updated edges to total edges in *it-2004* is smaller than those in *dblp-coauthor*, *yahoo-song*, and *sx-stackoverflow*.

• **Time cost of different components in HIT-Leiden.** There are three key components, i.e., *inc-movement*, *inc-refinement*, and *inc-aggregation*, in HIT-Leiden. We evaluate the proportion of time cost for each component and present the results in Fig. 15. Note that some operations (e.g., *def-update* in HIT-Leiden) may not be included by the above three components, so we put them into the "Others" component. Notably, in HIT-Leiden, the refinement phase contributes minimally to the overall runtime. Besides, the combined proportion of time spent in its movement and aggregation phases is comparable to that of other algorithms. *Inc-movement*, *inc-refinement*, and *inc-aggregation* in HIT-Leiden consistently outperform their counterparts in other algorithms across all datasets, achieving lower runtime costs according to Fig. 14.

• **Effect of b .** We vary the batch size $b \in \{10, 10^2, 10^3, 10^4, 10^5\}$ and report the efficiency in Fig. 16. We see that HIT-Leiden is up to five orders of magnitude faster than other algorithms. Also, its runtime decreases substantially as b

becomes smaller since it is a relatively bounded algorithm. In contrast, ND-Leiden, DS-Leiden, and DF-Leiden still need to process the entire graph when processing a new batch.

• **Effect of r .** Recall that after fixing the batch size b , we update the graph for r batches. Fig. 17 shows the efficiency, where b is fixed as 1,000, but r ranges from 1 to 9. We observe that the incremental speedup is limited in the first few batches because $P = 10$ is small, and additional iterations may slightly improve the community membership. As a result, all the maintenance algorithms often require more time for the second batch to adjust the community structure. Once high-quality community structure is established, the speedup becomes significant. In addition, HIT-Leiden incurs a slightly higher runtime at these first batches due to recording more information and constructing the CC-index.

VII. CONCLUSIONS

In this paper, we develop an efficient algorithm for maintaining Leiden communities in a dynamic graph. We first theoretically analyze the boundedness of existing algorithms and how supervertex behaviors affect community membership under graph update. Building on these analyses, we further develop a relatively bounded algorithm, called HIT-Leiden, which consists of three key components, i.e., *inc-movement*, *inc-refinement*, and *inc-aggregation*. Extensive experiments on five real-world dynamic graphs show that HIT-Leiden not only preserves the properties of Leiden and achieves comparable modularity quality with Leiden, but also runs faster than state-of-the-art competitors. In future work, we will extend our algorithm to handle directed graphs and also evaluate it in a parallel environment.

REFERENCES

- [1] M. E. Newman and M. Girvan, "Finding and evaluating community structure in networks," *Physical review E*, vol. 69, no. 2, p. 026113, 2004.
- [2] T. Chakraborty, A. Dalmia, A. Mukherjee, and N. Ganguly, "Metrics for community analysis: A survey," *ACM Computing Surveys (CSUR)*, vol. 50, no. 4, pp. 1–37, 2017.
- [3] X. Su, S. Xue, F. Liu, J. Wu, J. Yang, C. Zhou, W. Hu, C. Paris, S. Nepal, D. Jin *et al.*, "A comprehensive survey on community detection with deep learning," *IEEE transactions on neural networks and learning systems*, vol. 35, no. 4, pp. 4682–4702, 2022.
- [4] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *Journal of statistical mechanics: theory and experiment*, vol. 2008, no. 10, p. P10008, 2008.
- [5] V. A. Traag, L. Waltman, and N. J. Van Eck, "From Louvain to Leiden: guaranteeing well-connected communities," *Scientific reports*, vol. 9, no. 1, pp. 1–12, 2019.
- [6] Microsoft, "GraphRAG: A structured, hierarchical approach to retrieval augmented generation," <https://microsoft.github.io/graphrag/>, 2025, accessed: 2025-03-31.
- [7] L. Wang, H. Zhao, C. Feng, W. Liu, C. Huang, M. Santoni, M. Cristofaro, P. Jafrancesco, and J. Bian, "Removing camouflage and revealing collusion: Leveraging gang-crime pattern in fraudster detection," in *Proceedings of the 29th ACM SIGKDD conference on knowledge discovery and data mining*, 2023, pp. 5104–5115.
- [8] M. S. Haney, R. Pálovics, C. N. Munson, C. Long, P. K. Johansson, O. Yip, W. Dong, E. Rawat, E. West, J. C. Schlachetzki *et al.*, "ApoE4 is linked to damaging lipid droplets in alzheimer's disease microglia," *Nature*, vol. 628, no. 8006, pp. 154–161, 2024.
- [9] F. Liu, Z. Li, B. Wang, J. Wu, J. Yang, J. Huang, Y. Zhang, W. Wang, S. Xue, S. Nepal *et al.*, "eriskcom: an e-commerce risky community detection platform," *The VLDB Journal*, vol. 31, no. 5, pp. 1085–1101, 2022.
- [10] S. Sahu, "DF louvain: Fast incrementally expanding approach for community detection on dynamic graphs," *arXiv preprint arXiv:2404.19634*, 2024.
- [11] N. Zarayeneh and A. Kalyanaraman, "Delta-screening: a fast and efficient technique to update communities in dynamic graphs," *IEEE transactions on network science and engineering*, vol. 8, no. 2, pp. 1614–1629, 2021.
- [12] D. Zhuang, J. M. Chang, and M. Li, "Dynamo: Dynamic community detection by incrementally maximizing modularity," *IEEE Transactions on Knowledge and Data Engineering*, vol. 33, no. 5, pp. 1934–1945, 2019.
- [13] W. H. Chong and L. N. Teow, "An incremental batch technique for community detection," in *Proceedings of the 16th international conference on information fusion*. IEEE, 2013, pp. 750–757.
- [14] S. Sahu, K. Kothapalli, and D. S. Banerjee, "Towards dynamic community detection with leiden algorithm," in *2025 Supercomputing India (SCI)*. IEEE, 2025, pp. 1–8.
- [15] L. Xu, D. Wen, L. Qin, R. Li, Y. Zhang, and X. Lin, "Constant-time connectivity querying in dynamic graphs," *Proceedings of the ACM on Management of Data*, vol. 2, no. 6, pp. 1–23, 2024.
- [16] Z. Yang, R. Algesheimer, and C. J. Tessone, "A comparative analysis of community detection algorithms on artificial networks," *Scientific reports*, vol. 6, no. 1, p. 30750, 2016.
- [17] A. Amira, A. Derhab, E. B. Karbab, and O. Nouali, "A survey of malware analysis using community detection algorithms," *ACM Computing Surveys*, vol. 56, no. 2, pp. 1–29, 2023.
- [18] M. E. Newman, "Modularity and community structure in networks," *Proceedings of the national academy of sciences*, vol. 103, no. 23, pp. 8577–8582, 2006.
- [19] S. Kirkpatrick, C. D. Gelatt Jr, and M. P. Vecchi, "Optimization by simulated annealing," *science*, vol. 220, no. 4598, pp. 671–680, 1983.
- [20] M. E. Newman, "Spectral methods for community detection and graph partitioning," *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics*, vol. 88, no. 4, p. 042822, 2013.
- [21] F. Liu, J. Wu, C. Zhou, and J. Yang, "Evolutionary community detection in dynamic social networks," in *2019 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2019, pp. 1–7.
- [22] T. Zhang, Y. Xiong, J. Zhang, Y. Zhang, Y. Jiao, and Y. Zhu, "Commddgi: community detection oriented deep graph infomax," in *Proceedings of the 29th ACM international conference on information & knowledge management*, 2020, pp. 1843–1852.
- [23] L. Yang, X. Cao, D. He, C. Wang, X. Wang, and W. Zhang, "Modularity based community detection with deep learning," in *IJCAI*, vol. 16, no. 2016, 2016, pp. 2252–2258.
- [24] Y. Xie, M. Gong, S. Wang, and B. Yu, "Community discovery in networks with deep sparse filtering," *Pattern Recognition*, vol. 81, pp. 50–59, 2018.
- [25] T. Aynaud and J.-L. Guillaume, "Static community detection algorithms for evolving networks," in *8th international symposium on modeling and optimization in mobile, ad hoc, and wireless networks*. IEEE, 2010, pp. 513–519.
- [26] Y. Chi, X. Song, D. Zhou, K. Hino, and B. L. Tseng, "On evolutionary spectral clustering," *ACM Transactions on Knowledge Discovery from Data (TKDD)*, vol. 3, no. 4, pp. 1–30, 2009.
- [27] T. Aynaud and J.-L. Guillaume, "Multi-step community detection and hierarchical time segmentation in evolving networks," in *Proceedings of the 5th SNA-KDD workshop*, vol. 11, 2011.
- [28] J. Xie, M. Chen, and B. K. Szymanski, "Labelrank: Incremental community detection in dynamic networks via label propagation," in *Proceedings of the workshop on dynamic networks management and mining*, 2013, pp. 25–32.
- [29] M. Ester, H.-P. Kriegel, J. Sander, X. Xu *et al.*, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *kdd*, vol. 96, no. 34, 1996, pp. 226–231.
- [30] X. Xu, N. Yuruk, Z. Feng, and T. A. Schweiger, "SCAN: a structural clustering algorithm for networks," in *Proceedings of the 13th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2007, pp. 824–833.
- [31] S. Wang, Y. Fang, and W. Luo, "Searching and detecting structurally similar communities in large heterogeneous information networks," *Proceedings of the VLDB Endowment*, vol. 18, no. 5, pp. 1425–1438, 2025.
- [32] P. W. Holland, K. B. Laskey, and S. Leinhardt, "Stochastic blockmodels: First steps," *Social networks*, vol. 5, no. 2, pp. 109–137, 1983.
- [33] B. Karrer and M. E. Newman, "Stochastic blockmodels and community structure in networks," *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics*, vol. 83, no. 1, p. 016107, 2011.
- [34] E. M. Airoldi, D. Blei, S. Fienberg, and E. Xing, "Mixed membership stochastic blockmodels," *Advances in neural information processing systems*, vol. 21, 2008.
- [35] M. E. Newman, "Finding community structure in networks using the eigenvectors of matrices," *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics*, vol. 74, no. 3, p. 036104, 2006.
- [36] S. C. de Lange, M. A. de Reus, and M. P. van den Heuvel, "The laplacian spectrum of neural networks," *Frontiers in computational neuroscience*, vol. 7, p. 189, 2014.
- [37] R. Hu, S. Pan, G. Long, Q. Lu, L. Zhu, and J. Jiang, "Going deep: Graph convolutional ladder-shape networks," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 03, 2020, pp. 2838–2845.
- [38] G. Cui, J. Zhou, C. Yang, and Z. Liu, "Adaptive graph encoder for attributed graph embedding," in *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, 2020, pp. 976–985.
- [39] H. Zhao, X. Yang, Z. Wang, E. Yang, and C. Deng, "Graph debiased contrastive learning with joint representation clustering," in *IJCAI*, 2021, pp. 3434–3440.
- [40] X. Fu, J. Zhang, Z. Meng, and I. King, "Magnn: Metapath aggregated graph neural network for heterogeneous graph embedding," in *Proceedings of the web conference 2020*, 2020, pp. 2331–2341.
- [41] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," *arXiv preprint arXiv:1710.10903*, 2017.
- [42] X. Wang, N. Liu, H. Han, and C. Shi, "Self-supervised heterogeneous graph neural network with co-contrastive learning," in *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*, 2021, pp. 1726–1736.
- [43] B. Ruan, J. Gan, H. Wu, and A. Wirth, "Dynamic structural clustering on graphs," in *Proceedings of the 2021 International Conference on Management of Data*, 2021, pp. 1491–1503.
- [44] F. Zhang and S. Wang, "Effective indexing for dynamic structural graph clustering," *Proceedings of the VLDB Endowment*, vol. 15, no. 11, pp. 2908–2920, 2022.
- [45] Y. Chi, X. Song, D. Zhou, K. Hino, and B. L. Tseng, "Evolutionary spectral clustering by incorporating temporal smoothness," in *Proceedings of the 13th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2007, pp. 153–162.

- [46] C. Matias and V. Miele, “Statistical clustering of temporal networks through a dynamic stochastic block model,” *Journal of the Royal Statistical Society Series B: Statistical Methodology*, vol. 79, no. 4, pp. 1119–1141, 2017.
- [47] C. Yang, M. Liu, Z. Wang, L. Liu, and J. Han, “Graph clustering with dynamic embedding,” *arXiv preprint arXiv:1712.08249*, 2017.
- [48] N. Park, R. Rossi, E. Koh, I. A. Burhanuddin, S. Kim, F. Du, N. Ahmed, and C. Faloutsos, “Cgc: Contrastive graph clustering for community detection and tracking,” in *Proceedings of the ACM Web Conference 2022*, 2022, pp. 1115–1126.
- [49] Tencent, “Tencent graph computing (tgraph) officially open sourced high-performance graph computing framework: Plato,” https://github.com/Tencent/plato/blob/master/doc/introduction_en.md, 2019, accessed: 2026-04-17.
- [50] S. Sahu, K. Kothapalli, and D. S. Banerjee, “Fast leiden algorithm for community detection in shared memory setting,” in *Proceedings of the 53rd International Conference on Parallel Processing*, 2024, pp. 11–20.
- [51] J. Reichardt and S. Bornholdt, “Statistical mechanics of community detection,” *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics*, vol. 74, no. 1, p. 016110, 2006.
- [52] R. G. Lindeboom, K. B. Worlock, L. M. Dratva, M. Yoshida, D. Scobie, H. R. Wagstaffe, L. Richardson, A. Wilbrey-Clark, J. L. Barnes, L. Kretschmer *et al.*, “Human sars-cov-2 challenge uncovers local and systemic response dynamics,” *Nature*, vol. 631, no. 8019, pp. 189–198, 2024.
- [53] C. Lin, Y. Fang, Y. Xie, Y. Hu, Y. Hu, and C. Chen, “Maintaining leiden communities in large dynamic graphs (full version),” <https://github.com/swordzlim/HIT-Leiden>, 2026.
- [54] G. Ramalingam and T. Reps, “On the computational complexity of dynamic graph problems,” *Theoretical Computer Science*, vol. 158, no. 1-2, pp. 233–277, 1996.
- [55] W. Fan, C. Hu, and C. Tian, “Incremental graph computations: Doable and undoable,” in *Proceedings of the 2017 ACM International Conference on Management of Data*, 2017, pp. 155–169.
- [56] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. W. Cohen, R. Salakhutdinov, and C. D. Manning, “Hotpotqa: A dataset for diverse, explainable multi-hop question answering,” *arXiv preprint arXiv:1809.09600*, 2018.

APPENDIX A PROOF OF LEMMAS

A. Proof of Lemma 1

Proof. Assuming v changes its community from C_i to C , there are three cases:

- 1) For each neighbor v_i in C_i , the edge (v, v_i) is a *deleted intra-community* edge and an inserted cross-community edge;
- 2) For each neighbor v_j in C , the edge (v, v_j) is a deleted cross-community edge and an inserted intra-community edge;
- 3) For each other neighbor v_k , edge (v, v_k) is a deleted cross-community edge and an *inserted cross-community* edge.

Since only the first and third cases meet the conditions in Observation 1, all the neighbors of v that are not in C are likely to change their communities. $\square$

B. Proof of Lemma 2

Proof. We analyze the modularity gain $\Delta M(v \rightarrow \emptyset, \gamma)$ for any vertex v , which denotes the modularity gain of moving v from the intermediate subsequence I to $\emptyset$, whose calculation follows the same formula as the standard modularity gain.

According to Definition 8, if $\Delta M(v \rightarrow \emptyset, \gamma) > 0$, the intermediate subsequence I could not be γ -connected and v has to leave I . It is different from maintaining vertex optimality (mentioned in Definition 6): If there exists a community C' such that the modularity gain of moving v from its community

C to C' is positive, v is not locally optimized and has to be removed from C .

Case 1: v_i is inserted into S after v_j , i.e., $v_j \in I_i$. The old modularity gain $M_{old}(v_i \rightarrow \emptyset, \gamma) < 0$ before deletion is:

$$M_{old}(v_i \rightarrow \emptyset, \gamma) = -\frac{w(v_i, U_i)}{m} + \frac{\gamma \cdot d(v_i) \cdot d(U_i)}{2m^2} \leq 0. \quad (3)$$

Where $U_i = I_i \setminus \{v_i\}$. We multiply right side of (3) by $2m^2$ and obtain $X_{(3)}$:

$$X_{(3)} = -2m \cdot w(v_i, U_i) + \gamma \cdot d(v_i) \cdot d(U_i) \leq 0 \quad (4)$$

After the deletion, the new modularity gain $M_{new}(v_i \rightarrow \emptyset, \gamma)$ formulates:

$$\Delta M_{new}(v_i \rightarrow \emptyset, \gamma) = -\frac{w(v_i, U_i) - 2\alpha}{m - \alpha} + \frac{\gamma \cdot (d(v_i) - \alpha) \cdot (d(U_i) - \alpha)}{2(m - \alpha)^2}. \quad (5)$$

We multiply right side of (5) by $2(m - \alpha)^2$ and obtain $Y_{(5)}$:

$$\begin{aligned} Y_{(5)} &= -2(m - \alpha) \cdot (w(v_i, U_i) - 2\alpha) \\ &\quad + \gamma \cdot (d(v_i) - \alpha) \cdot (d(U_i) - \alpha) \\ &= X_{(3)} + \alpha \cdot (4m + 2w(v_i, U_i) - 4\alpha - \gamma \cdot (d(I_i) - \alpha)) \\ &< X_{(3)} + \alpha \cdot (4m + 2w(v_i, U_i)) \end{aligned} \quad (6)$$

If $X_{(3)} + \alpha \cdot (4m + 2w(v_i, U_i)) > 0$, $\Delta M_{new}(v_i \rightarrow \emptyset, \gamma)$ could be positive; Otherwise, $\Delta M_{new}(v_i \rightarrow \emptyset, \gamma)$ must be non-positive. Therefore, v_i could be removed from its sub-community only if $\alpha > \frac{2m \cdot w(v_i, U_i) - \gamma \cdot d(v_i) \cdot d(U_i)}{4m + 2w(v_i, U_i)}$.

Case 2: v_j is inserted into S before v_i . In this case, we have $v_j \in I_j$, $v_i \notin I_j$, and the edge deletion does not affect intra-edges within U_j . The old modularity gain $M_{old}(v_i \rightarrow \emptyset, \gamma) < 0$ before deletion is:

$$M_{old}(v_j \rightarrow \emptyset, \gamma) = -\frac{w(v_j, U_j)}{m} + \frac{\gamma \cdot d(v_j) \cdot d(U_j)}{2m^2}. \quad (7)$$

We multiply right side of (7) by $2m^2$ and obtain $X_{(7)}$:

$$X_{(7)} = -2m \cdot w(v_j, U_j) + \gamma \cdot d(v_j) \cdot d(U_j) < 0 \quad (8)$$

The new modularity gain after the edge deletion becomes:

$$\Delta M_{new}(v_j \rightarrow \emptyset, \gamma) = -\frac{w(v_j, U_j)}{m - \alpha} + \frac{\gamma \cdot (d(v_j) - \alpha) \cdot d(U_j)}{2(m - \alpha)^2} \quad (9)$$

We multiply right side of (9) by $2(m - \alpha)^2$ and obtain $Y_{(9)}$:

$$\begin{aligned} Y_{(9)} &= -2(m - \alpha) \cdot w(v_j, U_j) + \gamma \cdot (d(v_j) - \alpha) \cdot d(U_j) \\ &= X_{(7)} + 2\alpha \cdot (w(v_j, U_j) - \gamma \cdot d(U_j)) \\ &< X_{(7)} + 2\alpha \cdot w(v_j, U_j) \end{aligned} \quad (10)$$

Hence, v_j could be removed from its sub-community only if $\alpha > m - \frac{\gamma \cdot d(v_j) \cdot d(U_j)}{2w(v_j, U_j)}$.

Generalization to other vertices. Consider other vertices v_k and v_l such that $v_k \in S_i$, $k \neq i, j$ and $v_l \notin S_i$. The

old modularity gains $M_{old}(v_k \rightarrow \emptyset, \gamma) < 0$ and $M_{old}(v_l \rightarrow \emptyset, \gamma) < 0$ before deletion are:

$$M_{old}(v_k \rightarrow \emptyset, \gamma) = -\frac{w(v_k, U_k)}{m} + \frac{\gamma \cdot d(v_k) \cdot d(U_k)}{2m^2}. \quad (11)$$

$$M_{old}(v_l \rightarrow \emptyset, \gamma) = -\frac{w(v_l, U_l)}{m} + \frac{\gamma \cdot d(v_l) \cdot d(U_l)}{2m^2}. \quad (12)$$

We multiply right side of (11) and (12) by $2m^2$ respectively to obtain $X_{(11)}$ and $X_{(12)}$:

$$X_{(11)} = -2m \cdot w(v_k, U_k) + \gamma \cdot d(v_k) \cdot d(U_k) \leq 0 \quad (13)$$

$$X_{(12)} = -2m \cdot w(v_l, U_l) + \gamma \cdot d(v_l) \cdot d(U_l) \leq 0 \quad (14)$$

After the edge deletion, their new modularity gains are satisfied:

$$\Delta M_{new}(v_k \rightarrow \emptyset, \gamma) \leq -\frac{w(v_k, U_k)}{m - \alpha} + \frac{\gamma \cdot d(v_k) \cdot d(U_k)}{2(m - \alpha)^2}. \quad (15)$$

$$\Delta M_{new}(v_l \rightarrow \emptyset, \gamma) = -\frac{w(v_l, U_l)}{m - \alpha} + \frac{\gamma \cdot d(v_l) \cdot d(U_l)}{2(m - \alpha)^2}. \quad (16)$$

v_k could be merged before v_i and v_j , between v_i and v_j , as well as after v_i and v_j . $\Delta M_{new}(v_k \rightarrow \emptyset, \gamma)$ can be formulated as follows:

(1) v_k is merged before v_i and v_j :

$$\Delta M_{new}(v_k \rightarrow \emptyset, \gamma) = -\frac{w(v_k, U_k)}{m - \alpha} + \frac{\gamma \cdot d(v_k) \cdot d(U_k)}{2(m - \alpha)^2}; \quad (17)$$

(2) v_k is merged between v_i and v_j :

$$\Delta M_{new}(v_k \rightarrow \emptyset, \gamma) = -\frac{w(v_k, U_k)}{m - \alpha} + \frac{\gamma \cdot d(v_k) \cdot (d(U_k) - \alpha)}{2(m - \alpha)^2}; \quad (18)$$

(3) v_k is merged after v_i and v_j :

$$\Delta M_{new}(v_k \rightarrow \emptyset, \gamma) = -\frac{w(v_k, U_k)}{m - \alpha} + \frac{\gamma \cdot d(v_k) \cdot (d(U_k) - 2\alpha)}{2(m - \alpha)^2}. \quad (19)$$

Therefore, the equivalent of (15) holds if and only if v_k is merged before v_i and v_j . Then, We multiply right side of (15) and (16) by $2(m - \alpha)^2$ respectively and obtain $Y_{(15)}$ and $Y_{(16)}$:

$$\begin{aligned} Y_{(15)} &= -2(m - \alpha) \cdot w(v_k, U_k) \\ &\quad + \gamma \cdot d(v_k) \cdot d(U_k) \\ &= X_{13} + 2\alpha \cdot w(v_k, U_k), \end{aligned} \quad (20)$$

$$Y_{(16)} = X_{14} + 2\alpha \cdot w(v_l, U_l), \quad (21)$$

Only if $\alpha > m - \frac{\gamma \cdot d(v_k) \cdot d(U_k)}{2w(v_k, U_k)}$, v_k could be removed from its sub-community; v_l should be removed from its sub-community if and only if $\alpha > m - \frac{\gamma \cdot d(v_l) \cdot d(U_l)}{2w(v_l, U_l)}$. $\square$

C. Proof of Lemma 3

Proof. We adopt the same notations as in the proof of Lemma 2, with the exception that v_k now denotes a vertex residing in the same sub-community as either v_i or v_j . Based on this setup, the modularity gain after the edge deletion is shown as follows.

Case 1: Consider the endpoint v_i :

$$\begin{aligned} \Delta M_{new}(v_i \rightarrow \emptyset, \gamma) &= -\frac{w(v_i, U_i)}{m - \alpha} \\ &\quad + \frac{\gamma \cdot (d(v_i) - \alpha) \cdot d(U_i)}{2(m - \alpha)^2}. \end{aligned} \quad (22)$$

We multiply right side of (22) by $2(m - \alpha)^2$ and obtain $Y_{(22)}$:

$$\begin{aligned} Y_{(22)} &= -2(m - \alpha) \cdot w(v_i, U_i) \\ &\quad + \gamma \cdot (d(v_i) - \alpha) \cdot d(U_i) \\ &= X_{(3)} + \alpha \cdot (2w(v_i, U_i) - \gamma \cdot d(U_i)) \\ &< X_{(3)} + \alpha \cdot 2w(v_i, U_i) \end{aligned} \quad (23)$$

Only if $\alpha > m - \frac{\gamma \cdot d(v_i) \cdot d(U_i)}{2w(v_i, U_i)}$, v_i could be removed from its sub-community. v_j holds similar behavior.

Case 2: Consider the vertex $v_k \in S_i \cup S_j$, $k \neq i, j$:

$$\Delta M_{new}(v_k \rightarrow \emptyset, \gamma) \leq -\frac{w(v_k, U_k)}{m - \alpha} + \frac{\gamma \cdot d(v_k) \cdot d(U_k)}{2(m - \alpha)^2}. \quad (24)$$

For (24), v_k could be merged before v_i or v_j , as well as after v_i or v_j . Its equivalent holds if and only if v_k is merged before v_i or v_j . We multiply right side of (24) by $2(m - \alpha)^2$ and obtain $Y_{(24)}$:

$$\begin{aligned} Y_{(24)} &= -2(m - \alpha) \cdot w(v_k, U_k) + \gamma \cdot d(v_k) \cdot d(U_k) \\ &= X_{(11)} + 2\alpha \cdot w(v_k, U_k) \end{aligned} \quad (25)$$

Only if $\alpha > m - \frac{\gamma \cdot d(v_k) \cdot d(U_k)}{2w(v_k, U_k)}$, v_k could be removed from its sub-community.

Case 3: Consider the vertex $v_l \notin S_i \cup S_j$:

$$\Delta M_{new}(v_l \rightarrow \emptyset, \gamma) = -\frac{w(v_l, U_l)}{m - \alpha} + \frac{\gamma \cdot d(v_l) \cdot d(U_l)}{2(m - \alpha)^2}. \quad (26)$$

Similar to **Case 2**, if and only if $\alpha > m - \frac{\gamma \cdot d(v_l) \cdot d(U_l)}{2w(v_l, U_l)}$, v_l should be removed from its sub-community. $\square$

D. Proof of Lemma 4

Proof. First, we analyze the **insertion of intra-sub-community edges**. We adopt the same notations as in the proof of Lemma 2. Based on this setup, the modularity gain after the edge insertion is shown as follows.

Case 1: Consider the endpoint v_i , which is the latter merged endpoint:

$$\begin{aligned} \Delta M_{new}(v_i \rightarrow \emptyset, \gamma) &= -\frac{w(v_i, U_i) + 2\alpha}{m + \alpha} \\ &\quad + \frac{\gamma \cdot (d(v_i) + \alpha) \cdot (d(U_i) + \alpha)}{2(m + \alpha)^2}. \end{aligned} \quad (27)$$

We multiply right side of (27) by $2(m + \alpha)^2$ and obtain $Y_{(27)}$:

$$\begin{aligned} Y_{(27)} &= -2(m + \alpha)(w(v_i, U_i) + 2\alpha) \\ &\quad + \gamma \cdot (d(v_i) + \alpha) \cdot (d(U_i) + \alpha) \\ &= X_{(3)} + \alpha \cdot (\gamma \cdot (d(I_i) + \alpha) - 2w(v_i, U_i) - 4\alpha - 4m) \\ &< X_{(3)} + \alpha \cdot (\gamma \cdot (d(I_i) + \alpha) - 4m) \end{aligned} \quad (28)$$

Obviously, only if $\gamma \cdot (d(I_i) + \alpha) - 4m > 0$, i.e., $\alpha > \frac{4}{\gamma}m - d(I_i)$, $Y_{(27)}$ could be positive.

Case 2: Consider the endpoint v_j , which is the former merged endpoint:

$$\begin{aligned} \Delta M_{new}(v_j \rightarrow \emptyset, \gamma) &= -\frac{w(v_j, U_j)}{m + \alpha} \\ &\quad + \frac{\gamma \cdot (d(v_j) + \alpha) \cdot d(U_j)}{2(m + \alpha)^2}. \end{aligned} \quad (29)$$

We multiply right side of (29) by $2(m + \alpha)^2$ and obtain $Y_{(29)}$:

$$\begin{aligned} Y_{(29)} &= -2(m + \alpha) \cdot w(v_j, U_j) \\ &\quad + \gamma \cdot (d(v_j) + \alpha) \cdot d(U_j) \\ &= X_{(7)} + \alpha \cdot (\gamma \cdot d(U_j) - w(v_j, U_j)) \\ &< X_{(7)} + \alpha \cdot \gamma \cdot d(U_j) \end{aligned} \quad (30)$$

Only if $\alpha > \frac{2w(v_j, U_j)}{\gamma \cdot d(U_j)} \cdot m - d(v_j)$, v_j could be removed from its sub-community.

Case 3: Consider other vertex $v_k \in S_i, k \neq i, j$:

$$\begin{aligned} \Delta M_{new}(v_k \rightarrow \emptyset, \gamma) &\leq -\frac{w(v_k, U_k)}{m + \alpha} \\ &\quad + \frac{\gamma \cdot d(v_k) \cdot (d(U_k) + 2\alpha)}{2(m + \alpha)^2}. \end{aligned} \quad (31)$$

The equivalent of (31) holds if and only if v_k is merged after v_i and v_j . We multiply right side of (31) by $2(m + \alpha)^2$ and obtain $Y_{(31)}$:

$$\begin{aligned} Y_{(31)} &= -2(m + \alpha) \cdot w(v_k, U_k) \\ &\quad + \gamma \cdot d(v_k) \cdot (d(U_k) + 2\alpha) \\ &= X_{(11)} + \alpha \cdot (2\gamma \cdot d(v_k) - 2w(v_k, U_k)) \\ &< X_{(11)} + 2\alpha \cdot \gamma \cdot d(v_k) \end{aligned} \quad (32)$$

Only if $\alpha > \frac{w(v_k, U_k)}{\gamma \cdot d(v_k)} \cdot m - \frac{1}{2}d(U_k)$, v_k could be removed from its sub-community.

Case 4: Consider other vertex $v_l \notin S_i$:

$$\begin{aligned} \Delta M_{new}(v_l \rightarrow \emptyset, \gamma) &\leq -\frac{w(v_l, U_l)}{m + \alpha} \\ &\quad + \frac{\gamma \cdot d(v_l) \cdot d(U_l)}{2(m + \alpha)^2}. \end{aligned} \quad (33)$$

(33) holds if and only if v_l is merged after v_i and v_j . We multiply right side of (33) by $2(m + \alpha)^2$ and obtain $Y_{(33)}$:

$$\begin{aligned} Y_{(33)} &= -2(m + \alpha) \cdot w(v_l, U_l) \\ &\quad + \gamma \cdot d(v_l) \cdot d(U_l) \\ &= X_{(12)} - 2\alpha \cdot w(v_l, U_l) < 0 \end{aligned} \quad (34)$$

v_l is not affected by the intra-sub-community insertion.

Now, we consider the **insertion of cross-sub-community edges**. We adopt the same notations as in the proof of Lemma 3. Based on this setup, the modularity gain after the edge insertion is shown as follows.

Case 5: Consider the endpoint v_i :

$$\begin{aligned} \Delta M_{new}(v_i \rightarrow \emptyset, \gamma) &= -\frac{w(v_i, U_i)}{m + \alpha} \\ &\quad + \frac{\gamma \cdot (d(v_i) + \alpha) \cdot d(U_i)}{2(m + \alpha)^2}. \end{aligned} \quad (35)$$

We multiply right side of (35) by $2(m + \alpha)^2$ and obtain $Y_{(35)}$:

$$\begin{aligned} Y_{(35)} &= -2(m + \alpha) \cdot w(v_i, U_i) \\ &\quad + \gamma \cdot (d(v_i) + \alpha) \cdot d(U_i) \\ &= X_{(3)} + \alpha \cdot (\gamma \cdot d(U_i) - 2w(v_i, U_i)) \\ &< X_{(3)} + \alpha \cdot \gamma \cdot d(U_i) \end{aligned} \quad (36)$$

Only if $\alpha > \frac{2w(v_i, U_i)}{\gamma \cdot d(U_i)} \cdot m - d(v_i)$, v_i could be removed from its sub-community. v_j is the same.

Case 6: Consider other vertex $v_k \in S_i \cup S_j, k \neq i, j$:

$$\begin{aligned} \Delta M_{new}(v_k \rightarrow \emptyset, \gamma) &\leq -\frac{w(v_k, U_k)}{m + \alpha} \\ &\quad + \frac{\gamma \cdot d(v_k) \cdot (d(U_k) + \alpha)}{2(m + \alpha)^2}. \end{aligned} \quad (37)$$

The equivalent of (37) holds if and only if v_k is merged after v_i or v_j . We multiply right side of (37) by $2(m + \alpha)^2$ and obtain $Y_{(37)}$:

$$\begin{aligned} Y_{(37)} &= -2(m + \alpha) \cdot w(v_k, U_k) \\ &\quad + \gamma \cdot d(v_k) \cdot (d(U_k) + \alpha) \\ &= X_{(11)} + \alpha \cdot (\gamma \cdot d(v_k) - 2w(v_k, U_k)) \\ &< X_{(11)} + \alpha \cdot \gamma \cdot d(v_k) \\ &< X_{(11)} + 2\alpha \cdot \gamma \cdot d(v_k) \end{aligned} \quad (38)$$

v_k could be removed from its sub-community only if $\alpha > \frac{w(v_k, U_k)}{\gamma \cdot d(v_k)} \cdot m - \frac{1}{2}d(U_k)$.

Case 7: Consider other vertex $v_l \notin S_i \cup S_j$:

$$\begin{aligned} \Delta M_{new}(v_l \rightarrow \emptyset, \gamma) &\leq -\frac{w(v_l, U_l)}{m + \alpha} \\ &\quad + \frac{\gamma \cdot d(v_l) \cdot d(U_l)}{2(m + \alpha)^2}. \end{aligned} \quad (39)$$

(39) holds if and only if v_l is merged after v_i and v_j . We multiply right side of (39) by $2(m + \alpha)^2$ and obtain $Y_{(39)}$:

$$\begin{aligned} Y_{(39)} &= -2(m + \alpha) \cdot w(v_l, U_l) \\ &\quad + \gamma \cdot d(v_l) \cdot d(U_l) \\ &= X_{(12)} - 2\alpha \cdot w(v_l, U_l) < 0 \end{aligned} \quad (40)$$

v_l is not affected by the cross-sub-community insertion.

Conclusively, the effects of these edge insertions are:

- 1) v_i could be removed from its sub-community only if $\alpha > \frac{4}{\gamma}m - d(I_i)$ or $\alpha > \frac{2w(v_i, U_i)}{\gamma \cdot d(U_i)} \cdot m - d(v_i)$ according to **Case 1 and 5**.

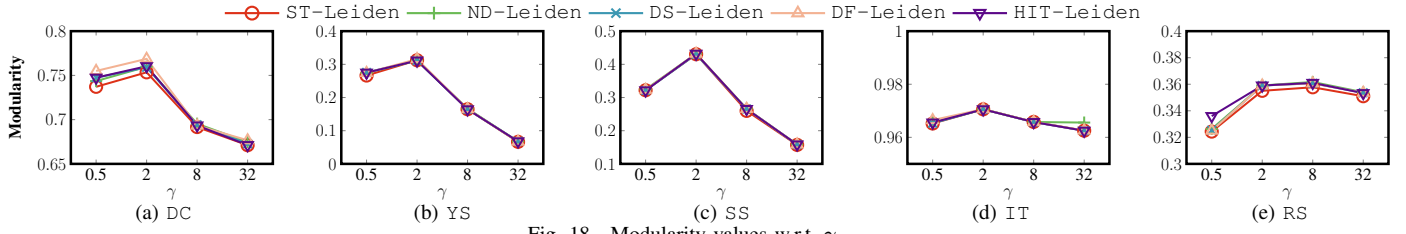


Fig. 18. Modularity values w.r.t. γ .

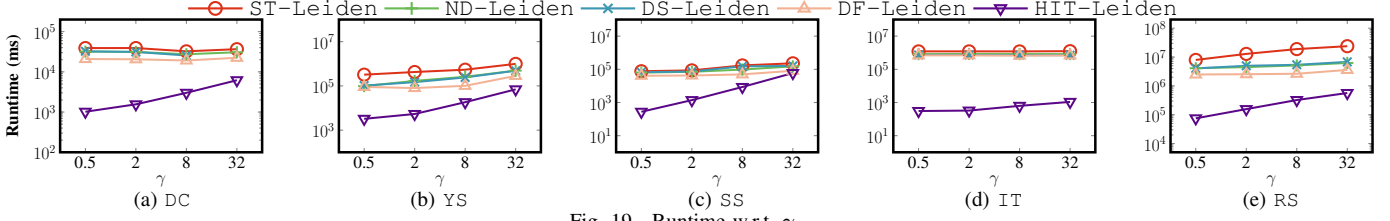


Fig. 19. Runtime w.r.t. γ .

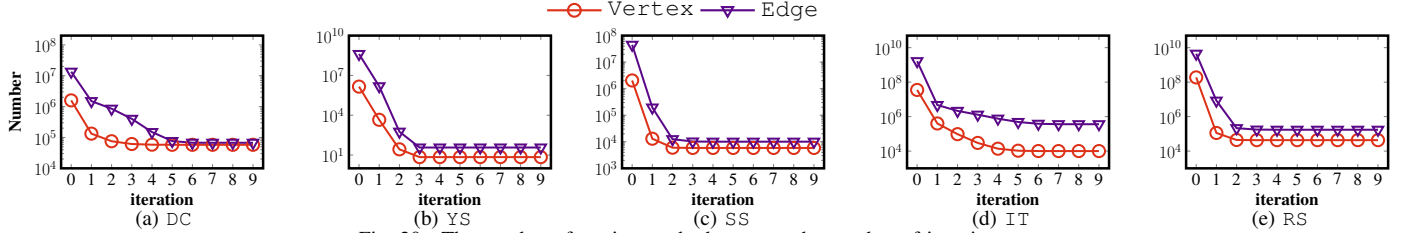


Fig. 20. The number of vertices and edges w.r.t. the number of iterations.

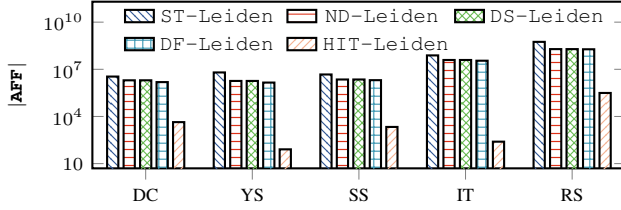


Fig. 21. Effect of $|AFF|$ on all datasets.

- 2) v_j could be removed from its sub-community, only if $\alpha > \frac{2w(v_j, U_j)}{\gamma \cdot d(U_j)} \cdot m - d(v_j)$ according to **Case 2 and 5**.
- 3) $v_k \in S_i \cup S_j$ ($k \neq i, j$) could be removed from its sub-community only if $\alpha > \frac{w(v_k, U_k)}{\gamma \cdot d(v_k)} \cdot m - \frac{1}{2} d(U_k)$ according to **Case 3 and 6**.
- 4) $v_l \notin S_i \cup S_j$ is unaffected according to **Case 4 and 7**.

□

APPENDIX B ADDITIONAL EXPERIMENTS

We now present our additional experimental results. First, we evaluate the effect of γ on modularity and runtime. Next, we show how the number of vertices and edges changes with the number of iterations. We then present the numerical results for AFF . Finally, we demonstrate the long-term validity of HIT-Leiden.

• **Effect of γ on modularity and runtime.** Fig. 18 shows the average modularity values for all maintenance algorithms, with the parameter $\gamma \in \{0.5, 2, 8, 32\}$ across all 9 batches, and with the batch size fixed at 1000. Across all datasets, our HIT-Leiden still achieves comparable modularity with other methods across different γ . Fig. 19 shows the average runtime across all maintenance algorithms under the identical configuration. Across all datasets, the runtime of HIT-Leiden

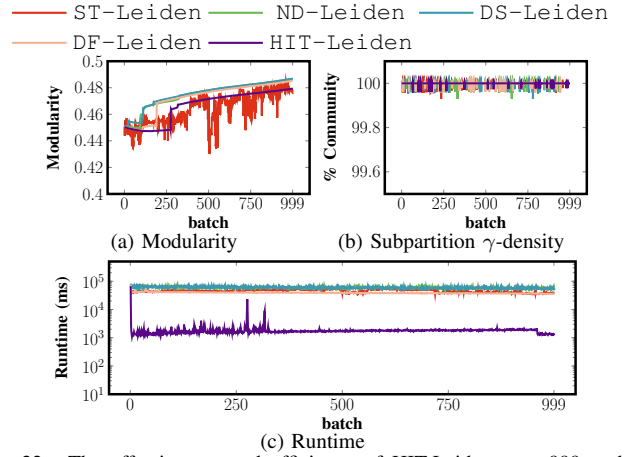


Fig. 22. The effectiveness and efficiency of HIT-Leiden over 999 update batches.

increases as γ increases. This occurs because higher γ values tend to form smaller, finer-grained communities, leading to an increase in the number of super-vertices that change their community or sub-community affiliation.

• Overall results for the number of vertices and edges.

Fig. 20 displays the overall results for the average number of vertices and edges in HIT-Leiden with respect to the number of iterations, with b and P set to their default values 1,000 and 10, respectively. Clearly, the first-level graph dominates the total size, and thus HIT-Leiden consumes asymptotically similar space to Leiden, whose space complexity is $O(|V| + |E|)$.

• Overall results for AFF .

Fig. 21 displays the overall results for the average number of supervertices that change their communities or sub-communities (i.e., $|AFF|$), with b set to its default value 1,000. Clearly, HIT-Leiden processes

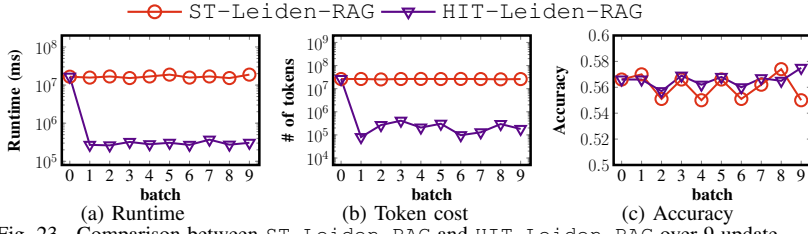


Fig. 23. Comparison between ST-Leiden-RAG and HIT-Leiden-RAG over 9 update batches on Graph-RAG.

the smallest affected region across datasets, particularly on *it-2004* dataset. This is consistent with the overall efficiency results in Fig. 14.

- **Long-term effectiveness.** To demonstrate the long-term effectiveness of maintaining communities, we enlarge the number r of batches from 9 to 999 and set $b = 10,000$. Fig. 22 presents the modularity, proportion of subpartition γ -dense communities, and runtime on the *sx-stackoverflow* dataset. We observe that incremental Leiden algorithms exhibit higher stability than ST-Leiden in modularity since they use previous community memberships, and HIT-Leiden is faster than other algorithms. Besides, HIT-Leiden presents its adaptability to substantial edge changes. Note that when updating the 276th batch, HIT-Leiden incurs high runtime due to substantial changes in community memberships caused by edge evolution, which is also reflected in the corresponding increase in modularity.

APPENDIX C TWO APPLICATION STUDIES

In this section, we apply HIT-Leiden to two real applications and analyze its empirical results.

- **Graph-RAG [6].** To improve the LLM generation for answering a question, people often first retrieve relevant information from an external corpus and then augment the question with this information before invoking the LLM. To facilitate retrieval, Graph-RAG builds an offline index: It first builds a graph for the corpus, then clusters the graph hierarchically using Leiden, and finally associates a summary for each community, which is generated by an LLM with some token cost. In practice, since the underlying corpus often changes, the communities and their summaries need to be updated as well. Our HIT-Leiden can not only dynamically update the communities efficiently, but also save the token cost since we only need to regenerate the summaries for the updated communities.

To experiment, we use the HotpotQA [56] dataset, which contains Wikipedia-based question-answer (QA) pairs. We randomly select 9,500 articles to build the initial graph, and insert 9 batches of new articles, each with 5 articles. The LLM we use is Doubao-1.5-pro-32k. To support a dynamic corpus, we adapt the static Graph-RAG method by updating communities using ST-Leiden and HIT-Leiden, respectively. These two RAG methods are denoted by ST-Leiden-RAG and HIT-Leiden-RAG, respectively. Note that ND-Leiden, DS-Leiden, and DF-Leiden are not fit to maintain the hierarchical communities of Graph-

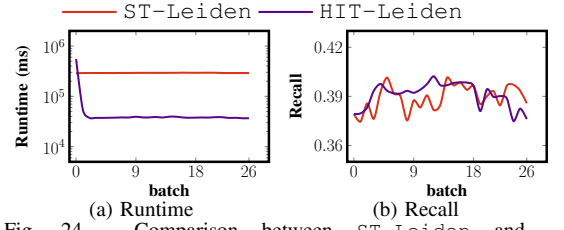


Fig. 24. Comparison between ST-Leiden and HIT-Leiden over 26 update batches on fraud-ring discovery.

RAG since they lack hierarchical maintenance. We report their runtime, token cost, and accuracy in Fig. 23. Clearly, HIT-Leiden-RAG is $56.1\times$ faster than ST-Leiden-RAG. Moreover, it significantly reduces the summary token cost while preserving downstream QA accuracy, since its token cost is only 0.8% of the token cost of ST-Leiden-RAG. Hence, HIT-Leiden is effective for supporting Graph-RAG on a dynamic corpus.

- **Fraud-ring discovery [9].** We also experiment with a workflow of fraud-ring discovery at ByteDance, which is different from the risk dataset used in the general maintenance benchmark in Table III. A fraud ring is a group with strong internal coordination that commits fraud. Many fraud cases are group-based, so fraud-ring discovery can be viewed as finding densely connected communities in a graph. Leiden partitions the graph into candidate communities, reducing search space and scaling efficiently. Based on Leiden communities, downstream modules produce a list of high-risk individuals. The list is then manually reviewed, and the recall of confirmed ring members is reported. As the graph evolves, we need to update Leiden communities to timely discover new potential ring members.

The input graph contains 58.3M vertices and 102.9M edges spanning 206 days, with daily updates (about 0.5M edge updates per batch). We first construct a static graph using the updates within the first 180 days. Then, we maintain the graph with a sliding window of 180 days; that is, we update it day by day by adding edges from the remaining 26 days while removing edges that fall outside the window. Finally, we evaluate the runtime of ST-Leiden and HIT-Leiden, since other processes are black-box to us, and the recall, as shown in Fig. 24. Clearly, HIT-Leiden achieves $7.7\times$ the speed of ST-Leiden with comparable recall.